

An Internship Report on
“AUTOMATED PODCAST TRANSCRIPTION
AND TOPIC SEGMENTATION”

BY

ADITI ARVIND CHINTAWAR

Under the Guidance
of
Mr. S. S. Neralkar

(Department of Computer Science and Engineering)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
Mahatma Gandhi Mission's College of Engineering, Nanded (M.S.)

Academic Year 2025-26

An Internship Report on

**“AUTOMATED PODCAST TRANSCRIPTION
AND TOPIC SEGMENTATION”**

Submitted to

**DR. BABASAHEB AMBEDKAR TECHNOLOGICAL
UNIVERSITY, LONERE**

for fulfillment of the requirement for the degree of

**BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING**

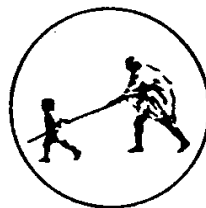
By

ADITI ARVIND CHINTAWAR

**Under the Guidance
of**

Mr. S. S. Neralkar

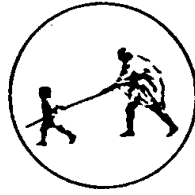
(Department of Computer Science and Engineering)



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MAHATMA GANDHI MISSION'S COLLEGE OF ENGINEERING
NANDED (M.S.)**

Academic Year 2025-26

Certificate



This is to certify that the Internship entitled

“AUTOMATED PODCAST TRANSCRIPTION AND TOPIC SEGMENTATION”

*being submitted by **MISS. ADITI ARVIND CHINTAWAR** to the Dr. Babasaheb Ambedkar Technological University, Lonere, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by her under my supervision and guidance. The matter contained in this report has not been submitted to any other university or institute for the award of any degree.*

Mr. S. S. Neralkar

Project Guide

Dr. A. M. Rajurkar

H.O.D

Computer Science & Engineering

Dr. G. S. Lathkar

Director

MGM's College of Engg, Nanded

INTERNSHIP OFFER LETTER



Infosys Springboard Virtual Internship 6.0 | You have been shortlisted!

1 message

Infosys Springboard-Support <Springboard-support@infosys.com>

Thu, 15 Jan, 2026 at 4:43 pm



Dear Learner,

Congratulations! You have been selected for the [Infosys Springboard Internship 6.0 Batch 13](#).

What's next?

- Watch out for upcoming emails with more information

Got a query? Write to springboard-support@infosys.com

Regards,
Team Infosys Springboard

Scan the QR code to download
the Infosys Springboard App



App Store



Play Store

Follow us on [Facebook](#) | [Instagram](#) | [LinkedIn](#) | [X \(Twitter\)](#)



Subject: Welcome to the Infosys Internship Program | Project Kick-off Session

1 message

springboard Mentor <springboardmentor22342d@gmail.com>
To: springboard Mentor <springboardmentor22342d@gmail.com>
Bcc: sd23_chintawar_aditi@mgmcen.ac.in

Tue, 3 Feb, 2026 at 7:10 pm

Dear Team,

Greetings.

I am pleased to introduce myself as your mentor, K Dikshita, an experienced Data Scientist. Welcome to the Infosys Internship Program, and congratulations on being selected. I look forward to working closely with you as we begin our journey on the project.

Kick-off Session

Our first virtual meeting is scheduled as follows:

Project Name: **'Automated Podcast Transcription and Topic Segmentation – Audio Analysis Project'**

Date: 5th February 2026

Time: 8:00 to 9:00 pm

Meeting Link: To join the meeting on Google Meet, click this link:

<https://meet.google.com/wnd-tjff-hbd>

Or open Meet and enter this code: wnd-tjff-hbd

(same link shall be used throughout the internship - if any changes, we will notify you)

Program Structure and Expectations

Please take note of the following important points:

1. From 5th February onwards, we will have daily sessions within the same time window. The exact timings can be discussed and finalized during the initial session, keeping everyone's feasibility in mind.
2. These sessions are not classroom-style lectures. They are intended for collaboration, guidance, and hands-on problem-solving, so there is no expectation of formal lectures.
3. The first few sessions will focus on:
 - 3.1. Introductions and alignment
 - 3.2. Understanding project objectives and deliverables
 - 3.3. Identifying any knowledge gaps
 - 3.4. Setting clear expectations for the internship
4. Attendance and Communication
 - 4.1. If you need to take leave at any point, please inform me in advance so that LEAVE can be marked in the attendance register. Otherwise, ABSENT will be recorded.
 - 4.2. For all official communication, please use email only to ensure clarity and proper documentation.
5. Professional Conduct
 - 5.1 All interactions are expected to maintain a high standard of professional decorum at all times.
 - 5.2. Respect for peers, mentor guidance, and the overall program structure is mandatory.

We hope this internship will help you develop strong problem-solving instincts and practical skills in data science through real-world project exposure.

I look forward to meeting all of you and starting this collaboration.

Warm regards,

K Dikshita

Mentor – Infosys Internship Program

 **AI_Automated_Podcast_Transcription (1).pdf**
212 KB



CERTIFICATE OF COMPLETION

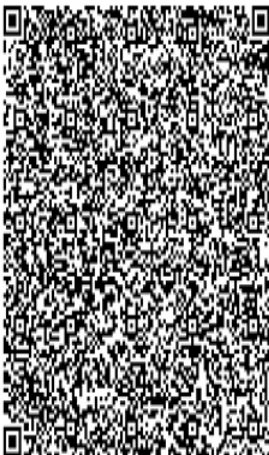
presented to

Aditi Chintawar

for actively participating and completing the mandatory assignment related to

Internship 6.0 (B 13): Automated Podcast Transcription with Topic Segmentation and Audio Analyzer

conducted from February 5, 2026 to April 3, 2026



Issued on: Thursday, May 28, 2026

To verify, scan the QR code at <https://verify.onwingspan.com>

Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my project guide, **Mr. S. S. Neralkar**, for his invaluable support, guidance, and encouragement throughout this project. His profound knowledge and expertise have been instrumental in the successful completion of this work. His patience and willingness to assist me at every step have greatly enriched my learning experience. His constructive feedback and insightful suggestions have not only helped me overcome challenges but also motivated me to strive for excellence.

I gladly take this opportunity to thank **Dr. A. M. Rajurkar** (Head of Computer Science & Engineering, MGM's College of Engineering, Nanded). We are heartily thankful to **Dr. G. S. Lathkar** (Director, MGM's College of Engineering, Nanded) for providing facilities during the progress of the project and also for her kind help, guidance and inspiration. Last but not least, we are also thankful to all those who helped, directly or indirectly, develop this project and complete it successfully.

With Deep Reverence,

ADITI ARVIND CHINTAWAR

[B.Tech CSE-B]

ABSTRACT

This report presents the development and implementation of the project titled “Automated Podcast Transcription and Topic Segmentation”, an AI-powered system designed to transform podcast audio into structured, meaningful, and easily navigable content. With the rapid growth of podcast platforms and long-form audio content, manually understanding and navigating podcast discussions has become time-consuming and inefficient. This project addresses these challenges by automating the transcription process and intelligently dividing podcast content into meaningful topic-based segments using Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques.

During the development of this project, I had the opportunity to apply my academic knowledge in the fields of Artificial Intelligence, Machine Learning, Natural Language Processing, Audio Processing, and Full-Stack Development to build an efficient and scalable solution. The system accepts podcast audio as input, preprocesses the audio to improve quality, and converts speech into text using the Faster Whisper speech recognition model. The generated transcript is then analyzed using NLP techniques to identify topic boundaries and segment the podcast into logical sections for better accessibility and understanding.

Throughout the project, I worked on both frontend and backend development using technologies such as Python, FastAPI, React, NLTK, SpaCy, SBERT, LibROSA, and PyDub. I implemented functionalities for audio preprocessing, transcript generation, semantic topic segmentation, keyword extraction, and automated summarization for each identified section. Beyond technical implementation, this project provided practical exposure to real-world problem solving, system integration, API development, and AI model optimization. Overall, this project successfully bridges the gap between lengthy audio content and user accessibility by providing an intelligent, structured, and time-efficient way to consume podcast information.

TABLE OF CONTENTS

TOPICS	PAGE NO.
INTERNSHIP OFFER LETTER	I
ACKNOWLEDGEMENT	IV
ABSTRACT	V
TABLE OF CONTENTS	VI
LIST OF FIGURES	IX
Chapter 1. INTRODUCTION	1
1.1 Internship Overview	2
1.2 Company Profile	2
1.3 Training and Skill Development	3
1.4 Projects and Work Experience	4
1.5 Learning Outcomes	6
1.6 Organization of the Report	8
Chapter 2. COMPANY BACKGROUND	10
2.1 Services Provided	10
2.2 Vision, Mission & Certifications	12
2.3 Community Engagement & SB Meetups	13
Chapter 3. TECHNOLOGY STACK	15
3.1 Python	16
3.1.1 Key Features of Python	17
3.1.2 Use Case in My Project	18
3.2 Natural Language Processing	19
3.2.1 Key Features of NLP Libraries	20
3.2.2 Application Workflow in My Internship	21
3.3 FasterWhisper	23
3.3.1 Key Features of FasterWhisper	24
3.3.2 Use Case in My Project	25

3.4 Audio Processing	26
3.4.1 Role of Audio Processing in Development	29
3.4.2 Use Case in My Project	30
3.5 Flask	31
3.5.1 Key Features of Flask	32
3.5.2 Application Workflow in My Project	33
3.5.3 Learning Outcomes and Benefits	34
Chapter 4. TECHNICAL TRAINING AND ASSIGNMENT	35
4.1 Python Programming and AI/ML Fundamentals	36
4.1.1 Core Python Concepts for Backend Development	37
4.1.2 Data Processing and Automation Concepts	38
4.1.3 AI/ML Concepts for Text Analysis	39
4.2 Backend Development with Flask	40
4.2.1 Setting Up the Development Environment and Flask Basics	41
4.2.2 RESTful API Development for Podcast Processing	42
4.2.3 Integration of AI Processing Pipeline	43
4.2.4 Handling Data Flow and Backend Logic	45
4.3 Frontend Integration and User Interface	46
4.3.1 Connecting Flask Backend with Frontend	48
4.3.2 Real-World Application and Dashboard	50
4.4 Version Control with Git and GitHub	51
4.4.1 Git Basics and Local Version Control	52
4.4.2 Collaborative Development Using GitHub	52
4.4.3 Real-World Development Experience	54
Chapter 5. INTERNSHIP PROJECT CONTRIBUTION	55
5.1 Initial Product Understanding and System Analysis	56
5.2 Frontend Console Cleanup and API Error Resolution	58
5.3 Backend Code Optimization Through Separation of Concerns	59
5.4 Middleware Implementation for Role-Based Access Control	60

5.5 Implementation of Role-Specific Functionality and Data Flow	61
5.6 Real-Time Chat Functionality Using Socket.IO	62
5.7 Report Functionality with Role-Based View Access	63
5.8 Styling and UI Enhancements Using Chakra UI and Utility Libraries	64
5.9 Code Optimization and Version Control Across Branches	65
5.10 Real-Time Bug Fixes and Feature Enhancements	66
CONCLUSION	68
REFERENCES	69

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
1.1	Infosys Springboard Logo	3
1.2	Workflow of Automated Podcast Transcription	5
2.1	Services Provided by Infosys Springboard	11
2.2	Vision and Mission of Infosys Springboard	13
3.1	Python Programming Language	16
3.2	Natural Language Processing	19
3.3	Faster Whisper	23
3.4	Audio Processing	27
4.1	Training Program Flow	36
5.1	System Home of Automated Podcast Analysis Website	57
5.2	Workflow of Podcast Processing and Dashboard	58
5.3	Podcast Upload, AI Processing and Data Flow Implementation	62
5.4	Real-Time Caption Generation and Chatbot Integration	63

Chapter 1

INTRODUCTION

Internships play a vital role in bridging the gap between theoretical academic learning and practical industry exposure. As part of my final semester of the B.Tech in Computer Science and Engineering program, I had the opportunity to undertake a virtual internship through Infosys Springboard, where I worked on the project titled “Automated Podcast Transcription and Topic Segmentation”. The internship served as an important platform for enhancing my technical capabilities and professional understanding while enabling me to develop an intelligent system capable of improving podcast accessibility and navigation. Infosys Springboard is a digital learning and virtual internship platform designed to equip students with industry-relevant technical knowledge and practical skills. The platform provides learning opportunities across various domains such as Artificial Intelligence, Machine Learning, Cybersecurity, Cloud Computing, Full-Stack Development, Data Science, and Software Engineering.

To prepare for the successful development of the project, I participated in a structured learning and skill development process during the internship period. This phase involved gaining practical knowledge of technologies and frameworks essential for developing an AI-powered application. I strengthened my understanding of Python programming, Natural Language Processing techniques, and audio processing methodologies. I also gained hands-on experience with tools and technologies such as NLTK, SpaCy, SBERT, LibROSA, PyDub, Faster Whisper, React, and FastAPI, which are widely used for intelligent content analysis, speech processing, and modern web application development. This learning phase helped me build a strong foundation for implementing the core functionalities of the proposed system.

During the internship, I applied the skills acquired through training and self-learning to design and develop the “Automated Podcast Transcription and Topic Segmentation” system. The project involved multiple stages of implementation, beginning with audio preprocessing to improve sound quality and reduce unwanted noise from podcast recordings. The processed audio was then converted into text using the Faster Whisper speech recognition model, enabling automatic transcript generation. Further, I implemented Natural Language Processing techniques to identify topic boundaries and divide long podcast discussions into meaningful segments. Additionally, I worked on keyword extraction, summarization, and frontend-backend integration to create a user-

friendly interface where users can upload podcast audio, access transcripts, view summaries, and navigate efficiently between discussion topics. This experience significantly improved my problem-solving abilities, programming expertise, and practical understanding of AI-driven system development.

1.1 Internship Overview

Internships play an important role in connecting academic learning with practical implementation by allowing students to gain exposure to real-world technologies, development practices, and problem-solving approaches. They provide opportunities to improve technical knowledge, strengthen professional skills, and understand how theoretical concepts can be applied to solve practical challenges. A virtual internship, in particular, offers flexibility while encouraging independent learning, innovation, and hands-on experience in emerging technological domains.

As part of my final semester of the B.Tech in Computer Science and Engineering program, I successfully completed a virtual internship through Infosys Springboard, where I worked on an Artificial Intelligence-based project titled “Automated Podcast Transcription and Topic Segmentation.” The internship focused on enhancing my understanding of Artificial Intelligence, Natural Language Processing, Audio Processing, and Full-Stack Development through practical implementation. Since podcasts often contain long and unstructured conversations, users face difficulty in finding relevant information quickly. Therefore, the project aimed to improve podcast accessibility through speech recognition, semantic analysis, keyword extraction, and automated summarization.

Throughout the internship period, I followed a systematic development approach that included learning relevant technologies, project planning, implementation, testing, and system integration. I worked on multiple project modules such as audio preprocessing, speech-to-text conversion using Faster Whisper, NLP-based topic segmentation, keyword extraction, summarization, and frontend-backend integration using React and FastAPI. In addition to technical development, the internship enhanced my analytical thinking, debugging skills, and ability to integrate multiple technologies into a single intelligent application. Overall, this experience significantly strengthened my technical foundation and prepared me for future opportunities in software development and AI-based systems.

1.2 Company Profile

Infosys Springboard is a digital learning and virtual internship platform developed by Infosys with the objective of empowering students, learners, and professionals through industry-relevant education and skill development. The platform provides access to a wide

range of learning resources, certification programs, and virtual internship opportunities across multiple domains, including Artificial Intelligence, Machine Learning, Data Science, Cybersecurity, Cloud Computing, Software Development, and Full-Stack Technologies. In today's rapidly evolving technological environment, acquiring practical exposure and hands-on experience has become essential for students to remain competitive in the professional world. Infosys Springboard addresses this requirement by creating an accessible digital ecosystem that promotes technical learning and career readiness.



Fig 1.1: Infosys Springboard Logo

The platform is designed to support continuous learning by offering structured training modules, real-world projects, assessments, and guided learning paths. The logo of Infosys Springboard represents innovation, growth, and knowledge-sharing, reflecting the organization's vision of making quality education and technical skill. Unlike traditional learning environments that focus primarily on theoretical knowledge, Infosys Springboard emphasizes practical implementation and experiential learning, enabling students to gain exposure to modern technologies and solve real-world challenges through project-based approaches.

Infosys Springboard distinguishes itself through its learner-centric approach, industry-oriented curriculum, and focus on emerging technologies. The platform encourages self-paced learning while also promoting innovation, analytical thinking, and practical problem-solving abilities. During my internship period, Infosys Springboard provided me with the opportunity to work on the project titled "Automated Podcast Transcription and Topic Segmentation," which enabled me to explore technologies such as Python, Natural Language Processing (NLP), Audio Processing, React, and FastAPI.

1.3 Training and Skill Development

An important aspect of my internship was the learning and skill development phase, which played a crucial role in preparing me for the successful implementation of my project. Before beginning the actual development work, I focused on understanding the core technologies, concepts, and methodologies required to build an intelligent system for

podcast transcription and topic segmentation. This phase was designed to strengthen both theoretical understanding and practical implementation skills related to Artificial Intelligence, Natural Language Processing (NLP), Audio Processing, and Full-Stack Development.

During the training period, I significantly strengthened my understanding of Python programming, which served as the primary programming language for project development. I explored important Python concepts such as functions, modules, file handling, exception handling, object-oriented programming, and library integration. Additionally, I learned to work with multiple Python libraries and frameworks to improve system performance and automation.

A major focus of the training involved gaining practical knowledge of Natural Language Processing (NLP) techniques and tools. I worked with libraries such as NLTK, SpaCy, and SBERT, which enabled me to understand text preprocessing, tokenization, stop-word removal, semantic similarity analysis, and topic identification.

Since the project was based on podcast analysis, understanding audio preprocessing techniques became essential for improving transcription quality. I worked with tools such as LibROSA and PyDub for handling audio cleaning, format conversion, and preprocessing tasks. Additionally, I learned to use the Faster Whisper speech recognition model, which enabled efficient and accurate conversion of podcast audio into textual transcripts. Understanding how speech recognition models function and optimizing their performance helped improve the overall efficiency of the system.

Along with backend intelligence, I also gained practical exposure to frontend and backend application development. For frontend implementation, I explored React to create an interactive and user-friendly interface that enables users to upload podcast audio, access transcripts, and navigate through segmented topics efficiently. On the backend side, I worked with FastAPI for handling API requests, integrating AI functionalities, and enabling smooth communication between different modules of the system. This training and skill development phase laid a strong technical foundation and significantly contributed to the successful completion of the project.

1.4 Projects and Work Experience

Following the training and skill development phase, I transitioned into the implementation of my internship project titled “Automated Podcast Transcription and Topic Segmentation.” The project focused on solving the practical challenge of understanding and navigating long-duration podcast content efficiently. Since podcasts often contain

lengthy and unstructured discussions, users face difficulty locating relevant information or revisiting important conversations. Therefore, the project was designed to automate podcast transcription and intelligently divide podcast discussions into meaningful topic-based sections using Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques. knowledge in audio processing, NLP, and full-stack application development.

On the backend side, I worked on implementing the core functionalities required for processing podcast audio and generating meaningful outputs. This included audio preprocessing, speech-to-text conversion, transcript analysis, topic segmentation, keyword extraction, and summarization. I utilized tools such as LibROSA and PyDub to preprocess podcast audio by improving sound quality and preparing files for accurate transcription. For speech recognition, I integrated the Faster Whisper model, which enabled efficient and accurate conversion of podcast audio into textual transcripts. Once the transcript was generated, I applied NLP techniques using NLTK, SpaCy, and SBERT to analyze sentence-level semantic relationships and identify meaningful topic boundaries. Additionally, I implemented keyword extraction and summary generation functionalities to help users quickly understand the key points discussed in different podcast sections.

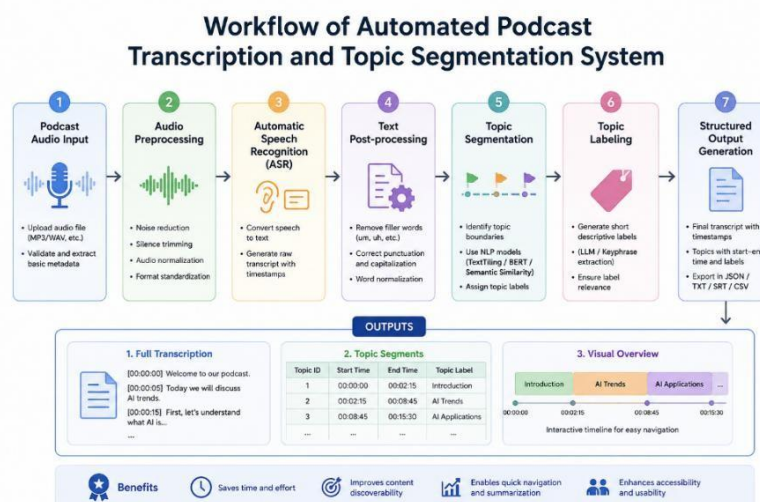


Fig 1.2: Workflow of Automated Podcast Transcription

In addition to technical implementation, the project involved testing, debugging, and performance optimization to improve overall system efficiency and reliability. I continuously evaluated different approaches for topic segmentation and transcript quality enhancement to improve system accuracy. Overall, the internship project provided me with valuable practical experience and strengthened my confidence in developing intelligent, scalable, and user-oriented applications using modern technologies.

1.5 Learning Outcomes

The virtual internship through Infosys Springboard proved to be a highly valuable and transformative learning experience that significantly contributed to my technical, analytical, and professional growth. Through the successful development of the project titled “Automated Podcast Transcription and Topic Segmentation”, I gained practical exposure to the real-world implementation of advanced technologies such as Artificial Intelligence (AI), Natural Language Processing (NLP), Speech Recognition, Audio Processing, Machine Learning, and Full-Stack Web Development.

One of the major learning outcomes of this internship was gaining hands-on experience in speech-to-text technology using OpenAI Whisper AI. I learned how audio data is processed and converted into accurate textual transcriptions while handling challenges such as varying speech patterns, background noise, speaker clarity, and large audio files. This practical implementation enhanced my understanding of audio processing pipelines and real-time transcription systems.

In addition, I developed strong knowledge of Natural Language Processing (NLP) techniques, particularly in topic segmentation, sentiment analysis, summarization, and keyword extraction. I learned how NLP models can be used to extract meaningful insights from large amounts of unstructured text data. Working on transcript analysis improved my understanding of text preprocessing, tokenization, semantic understanding, and contextual information extraction using AI-based techniques.

The internship also strengthened my programming skills in Python, which was extensively used for backend logic, data processing, API integration, and AI model implementation. I gained practical exposure to multiple Python libraries and frameworks used in AI development, including Flask, Whisper, Transformers, TextBlob, NLTK, Scikit-learn, Pandas, and Matplotlib. This helped me understand how different technologies work together to build a complete intelligent system.

Another important learning outcome was gaining practical knowledge of full-stack web application development. I worked on integrating the frontend and backend components of the system to create an interactive and user-friendly web platform. This involved designing dashboards, handling user inputs, integrating APIs, managing data flow, and displaying analytical insights through visualizations such as graphs, charts, and word clouds. I also learned how to improve user experience by implementing responsive interfaces and interactive features.

Through the implementation of the AI-powered chatbot functionality, I gained an understanding of conversational AI and retrieval-based response generation. I learned how chatbot systems process user queries, retrieve relevant contextual information, and provide meaningful answers related to podcast content. This enhanced my knowledge of intelligent assistance systems and real-time information retrieval.

The internship also provided exposure to project management and software development practices, including requirement analysis, debugging, testing, problem-solving, and performance optimization. I learned the importance of writing clean, maintainable, and modular code while ensuring system reliability and efficiency. Handling technical challenges during project implementation improved my troubleshooting abilities and analytical thinking.

Beyond technical skills, the internship significantly contributed to my professional and interpersonal development. It enhanced my communication skills, teamwork, time management, self-learning ability, and problem-solving mindset. Since the internship required continuous learning and independent implementation of technologies, it improved my confidence in adapting to new tools and emerging technologies.

Furthermore, I developed a better understanding of how Artificial Intelligence can be used to automate real-world tasks and improve accessibility. The project demonstrated how AI can simplify content analysis, save user time, and generate valuable insights from audio-based information sources such as podcasts.

Additionally, this internship helped me understand the importance of continuous learning and adaptability in the rapidly evolving field of technology. It motivated me to explore advanced concepts in AI-driven systems, automation, and intelligent application development, thereby strengthening my interest in research and innovation.

Overall, this internship provided me with a strong technical foundation and practical industry-oriented experience that will greatly support my future academic and professional career. It enhanced my confidence in working on real-world software projects and strengthened my ability to independently design, develop, and deploy intelligent applications. The knowledge and experience gained during this internship will continue to guide my growth as a software engineer and AI enthusiast, preparing me for future challenges and career opportunities in emerging technologies.

The virtual internship through Infosys Springboard has been a highly valuable and enriching experience that significantly contributed to my technical, analytical, and professional development. Through the successful implementation of the project “Automated Podcast Transcription and Topic Segmentation,” I gained practical exposure to the real-world application of Artificial Intelligence (AI), Natural Language Processing (NLP), Audio Processing, and Full-Stack Development.

Overall, this internship provided me with a strong foundation for my future career in software development and Artificial Intelligence-based applications. It enhanced my confidence in working on real-world technical projects and improved my ability to learn and implement emerging technologies independently. The experience gained through this internship will continue to support my academic and professional growth, and I look forward to applying these learnings in future projects and career opportunities.

1.6 Organization of the Report

This report is structured into five chapters, each covering different aspects of the internship, project development process, technologies used, implementation methodology, and outcomes achieved during the virtual internship through Infosys Springboard.

Chapter 1: Introduction – This chapter provides an overview of the virtual internship, project objectives, company profile, training and skill development, project implementation, and key learning outcomes gained during the internship. It also introduces the project titled “Automated Podcast Transcription and Topic Segmentation” and explains its significance in solving real-world podcast accessibility challenges.

Chapter 2: Literature Review and Problem Background – This chapter explains the existing system, limitations of traditional podcast navigation methods, and the need for an intelligent transcription and topic segmentation system. It also discusses the problem statement, objectives of the proposed system, and the relevance of Artificial Intelligence and Natural Language Processing in podcast content analysis.

Chapter 3: Technology Stack – This chapter highlights the major technologies, programming languages, frameworks, and tools used during the project development process. It includes detailed information about Python, NLTK, SpaCy, SBERT, Faster Whisper, LibROSA, PyDub, React, and FastAPI, along with their roles and applications in implementing the project functionalities.

Chapter 4: System Design and Project Implementation – This chapter describes the architectural design, workflow, and implementation of the project. It explains important modules such as audio preprocessing, speech-to-text conversion, transcript generation,

NLP-based topic segmentation, keyword extraction, summarization, and frontend-backend integration. The chapter also discusses system functionality and implementation strategies adopted during development.

Chapter 5: Results, Challenges, and Learning Outcomes – This chapter presents the final project outcomes, system performance, challenges encountered during development, solutions implemented, and practical knowledge gained throughout the internship. It also discusses the overall contribution of the project and future scope for further enhancement of the system.

The report has been prepared in a systematic manner to provide a comprehensive understanding of both the internship experience and the technical implementation of the project. Each chapter builds upon the previous one, allowing readers to gradually understand the problem domain, technological foundation, development methodology, and practical outcomes achieved during the internship period. This structured approach ensures clarity, readability, and effective presentation of the work carried out throughout the project lifecycle.

Furthermore, the report serves as a detailed record of the knowledge, skills, and industry-oriented experience gained through the Infosys Springboard Virtual Internship Program. It highlights the practical application of theoretical concepts learned during academic studies and demonstrates the ability to design, develop, and deploy an Artificial Intelligence-based solution for real-world challenges. The document also provides valuable insights for future researchers and students interested in the fields of Natural Language Processing, Speech Recognition, Audio Analytics, and Intelligent Content Management Systems.

Chapter 2

COMPANY BACKGROUND

Infosys Springboard is an innovative digital learning and virtual internship initiative developed by Infosys with the objective of empowering students, educators, and lifelong learners through accessible, industry-oriented education. The platform has been designed to bridge the gap between academic knowledge and practical industry requirements by providing learning resources, certification courses, and project-based internships in emerging technological domains. In today's rapidly evolving digital era, where technology continues to transform industries, acquiring practical knowledge and technical expertise has become essential for students and professionals. Infosys Springboard addresses this need by creating an ecosystem that promotes continuous learning, skill enhancement, and career readiness.

Since its inception, Infosys Springboard has focused on building an inclusive and learner-centric educational environment that enables individuals to gain exposure to real-world technologies and industry practices. The platform offers courses and internship opportunities across multiple domains, including Artificial Intelligence, Machine Learning, Cybersecurity, Cloud Computing, Data Science, Full-Stack Development, Software Engineering, Business Communication, and Digital Technologies. Unlike traditional learning approaches that focus mainly on theoretical understanding, Infosys Springboard emphasizes experiential learning through hands-on projects, guided exercises, assessments, and practical implementation.

Infosys, the organization behind Springboard, is globally recognized for its contribution to information technology services, consulting, digital transformation, and innovation. Infosys Springboard reflects this commitment by offering structured learning paths, mentorship opportunities, and industry-relevant projects that help learners prepare for professional careers. Through this platform, I gained valuable practical exposure, strengthened my technical foundation, and developed confidence in applying theoretical concepts to real-world software development and Artificial Intelligence-based applications.

2.1 Services Provided

Infosys Springboard provides a comprehensive range of educational, technical, and career-oriented services designed to help students, professionals, and lifelong learners acquire

industry-relevant skills. The platform focuses on improving employability and technical competency by offering structured learning resources, certification programs, virtual internships, and project-based learning opportunities. In today's rapidly evolving technological environment, where industries demand practical skills alongside academic knowledge, Infosys Springboard serves as an important platform for learners to strengthen both theoretical understanding and practical implementation capabilities.

One of the major services provided by Infosys Springboard is its online learning and certification ecosystem. The platform offers a diverse collection of courses across domains such as Artificial Intelligence, Machine Learning, Data Science, Cybersecurity, Cloud Computing, Full-Stack Development, Programming Languages, Business Communication, and Software Engineering. Additionally, learners receive certifications after successful completion of courses, which helps strengthen their professional profiles and improve career readiness.



Fig 2.1: Services Provided by Infosys Springboard

Another important service offered by Infosys Springboard is virtual internships and project-based learning opportunities. Through practical projects, students gain exposure to software development methodologies, problem-solving strategies, and technical implementation skills. During my internship, I had the opportunity to work on the project titled “Automated Podcast Transcription and Topic Segmentation,” which enabled me to apply concepts related to Artificial Intelligence, Natural Language Processing (NLP), Audio Processing, and Full-Stack Development in a practical environment. Such project experiences significantly contribute to improving technical proficiency and confidence in handling real-world challenges.

In addition to technical training, Infosys Springboard also offers services related to career development, soft skill enhancement, and digital learning support. The platform

provides resources focused on communication skills, leadership, teamwork, professional ethics, and workplace readiness, helping students prepare for future career opportunities. Furthermore, the self-paced learning environment allows learners to access educational content according to their convenience, making quality learning more accessible and flexible. Through its combination of technical education, internships, certifications, and career-oriented resources, Infosys Springboard plays an important role in empowering students with the knowledge and practical experience required to succeed in the modern digital world.

2.2 Vision, Mission & Certifications

At the core of Infosys Springboard lies a strong vision of empowering individuals through accessible, high-quality digital learning and skill development opportunities. The platform aims to bridge the gap between academic education and industry requirements by providing students and professionals with practical exposure to emerging technologies and industry-oriented knowledge. Infosys Springboard believes that education should not be limited by geographical, financial, or social barriers. Instead, learning should be inclusive, flexible, and designed to help individuals develop skills that improve employability and career readiness in a rapidly changing technological world.

Complementing this vision is the platform's mission to create a digitally skilled workforce by offering structured learning resources, certification programs, and practical internship opportunities across multiple domains. Infosys Springboard is committed to helping learners strengthen both technical and professional competencies by focusing on real-world problem solving, innovation, and experiential learning. Through guided learning modules and project-based internships, the platform encourages students to apply theoretical concepts in practical scenarios, thereby improving analytical thinking and technical implementation abilities. During my internship, this mission enabled me to work on the project titled "Automated Podcast Transcription and Topic Segmentation," which provided practical exposure to Artificial Intelligence, Natural Language Processing, Audio Processing, and Full-Stack Development.

The internship experience through Infosys Springboard not only enhanced my technical knowledge but also contributed significantly to my overall professional development. It provided me with an opportunity to work in a structured and goal-oriented environment, where I learned the importance of time management, problem-solving strategies, and collaborative development practices. By engaging in hands-on project work, I was able to understand how industry-level applications are designed, developed, and

optimized. This experience also improved my confidence in handling complex systems and integrating multiple technologies into a single solution. Overall, the internship played a crucial role in preparing me for future career opportunities in the field of Artificial Intelligence and software development by equipping me with both practical skills and industry-relevant experience.



Fig 2.2: Vision and Mission of Infosys Springboard

The vision, mission, and educational initiatives of Infosys Springboard collectively reflect the organization's commitment to creating future-ready professionals. Rather than focusing solely on theoretical learning, the platform emphasizes practical skill development, critical thinking, and continuous learning. By offering accessible education, project-based internships, and technology-focused training, Infosys Springboard helps students build strong technical foundations and confidence in solving real-world challenges. The internship experience provided through this platform significantly contributed to my technical growth and strengthened my understanding of modern software development and Artificial Intelligence-based applications.

2.3 Community Engagement & SB Meetups

Community engagement and collaborative learning form an important part of the educational philosophy of Infosys Springboard. The platform strongly believes that learning should not be restricted to classrooms or theoretical concepts alone, but should extend toward practical exposure, collaboration, and continuous skill enhancement.

Another important aspect of Infosys Springboard is its role in supporting career growth and talent development among students and aspiring professionals. Through project-based internships and skill-building programs, learners are encouraged to explore innovative ideas, strengthen problem-solving abilities, and gain confidence in

implementing practical solutions. During my internship, I had the opportunity to work on the project titled “Automated Podcast Transcription and Topic Segmentation,” which allowed me to apply concepts related to Artificial Intelligence, Natural Language Processing, Audio Processing, and Full-Stack Development in a practical environment. This experience not only improved my technical skills but also enhanced my confidence in handling real-world software development challenges independently.

In addition to technical learning, the community engagement activities conducted through Infosys Springboard encouraged participants to interact with learners from diverse educational and professional backgrounds. These interactions created opportunities for knowledge sharing, collaborative problem-solving, and discussions on emerging technologies such as Artificial Intelligence, Machine Learning, Cloud Computing, Cybersecurity, and Data Science. Through various webinars, expert sessions, and virtual meetups, participants were able to gain industry insights and understand the practical applications of modern technologies in solving real-world challenges. Such engagements helped bridge the gap between academic learning and industry expectations.

The SB Meetups and community-driven events also contributed significantly to the development of professional and interpersonal skills. Regular participation in discussions, technical presentations, and collaborative activities enhanced communication abilities, teamwork, leadership qualities, and confidence in expressing technical ideas. These experiences complemented the technical training received during the internship and provided a holistic learning environment that focused on both personal and professional growth. As a result, learners were better prepared to adapt to dynamic workplace environments and contribute effectively to future software development and technology-driven projects.

In conclusion, Infosys Springboard plays an important role in empowering learners by combining quality education, practical implementation, and career-focused development opportunities. Its learner-centric approach, emphasis on emerging technologies, and commitment to accessible education make it a valuable platform for students aiming to strengthen their technical foundation. The virtual internship experience significantly contributed to my academic and professional growth, helping me develop technical expertise, analytical thinking, and practical problem-solving abilities that will support my future career in software development and Artificial Intelligence-based systems.

Chapter 3

TECHNOLOGY STACK

In any software development project, selecting an appropriate technology stack is one of the most important factors that influences the overall efficiency, scalability, performance, and maintainability of the system. The term “technology stack” refers to the combination of programming languages, frameworks, libraries, tools, and technologies used during the development of a software application. A carefully selected technology stack enables developers to build efficient, reliable, and user-friendly systems capable of solving real-world challenges effectively. It also helps in reducing development complexity, improving system integration, and ensuring better performance throughout the software lifecycle.

For the development of the project titled “Automated Podcast Transcription and Topic Segmentation,” a combination of modern technologies related to Artificial Intelligence (AI), Natural Language Processing (NLP), Audio Processing, and Web Development was utilized. Since the project involves processing podcast audio, converting speech into text, identifying meaningful topic boundaries, and presenting structured information to users, the technology stack was selected to efficiently support these functionalities. The primary programming language used for the project was Python, due to its flexibility, extensive library support, and strong capabilities in Artificial Intelligence and data processing applications.

To enable intelligent transcript generation and semantic analysis, multiple specialized technologies and libraries were integrated into the system. Faster Whisper was used as the speech recognition model to convert podcast audio into textual transcripts with improved speed and accuracy. For Natural Language Processing tasks such as text preprocessing, semantic similarity analysis, keyword extraction, and topic segmentation, libraries including NLTK, SpaCy, and SBERT were utilized. Additionally, LibROSA and PyDub played an important role in audio preprocessing by improving audio quality, handling file transformations, and preparing podcast recordings for accurate transcription. These technologies collectively helped improve the efficiency and accuracy of the podcast analysis process.

To support backend functionality and user interaction, Flask was used as the web framework for connecting different system modules and managing application workflows. Flask enabled smooth handling of file uploads, API integration, data processing, and

communication between different components of the project. For frontend implementation, technologies such as HTML, CSS, and JavaScript were used to develop a simple, interactive, and user-friendly interface that allows users to upload podcast files, access transcripts, and navigate through segmented podcast topics conveniently. The combination of frontend and backend technologies ensured smooth system functionality and improved overall usability.

In addition to simplifying development, the selected technology stack supported a modular and scalable project structure. Each technology complemented the others by addressing specific system requirements, such as audio processing, transcription, semantic analysis, backend communication, and frontend interaction. The integration of these technologies enabled efficient system performance, reduced processing complexity, and ensured accurate output generation. Overall, the chosen technology stack played a crucial role in the successful implementation of the project by providing a strong technical foundation for developing an intelligent, efficient, and user-oriented podcast transcription and topic segmentation system.

3.1 Python

Python is a high-level, open-source, and widely used programming language known for its simplicity, readability, and versatility. Developed by Guido van Rossum, Python has become one of the most popular programming languages for software development, web applications, data analysis, Artificial Intelligence (AI), Machine Learning, and Natural Language Processing (NLP). Its easy-to-understand syntax and extensive collection of libraries make it an ideal choice for developing complex systems efficiently while reducing development time and coding complexity. Python supports multiple programming paradigms, including object-oriented, procedural, and functional programming, making it highly flexible for different application requirements.



Fig 3.1: Python Programming Language

which served as the primary programming language for the development of the

“Automated Podcast Transcription and Topic Segmentation” project. One of the major reasons for selecting Python was its strong ecosystem of libraries and frameworks that support Artificial Intelligence, audio processing, and text analysis. Unlike traditional programming languages that may require extensive coding for complex operations, Python provides pre-built libraries that simplify tasks such as speech recognition, semantic text processing, data handling, and backend integration. This significantly improved development efficiency and enabled faster implementation of project functionalities.

The use of Python significantly improved the scalability, efficiency, and maintainability of the project. Its clean syntax made code implementation easier to understand, debug, and optimize during development. Furthermore, Python’s extensive community support and continuous updates ensured access to modern tools and resources required for solving technical challenges. Overall, Python served as the backbone of the project by enabling the successful integration of Artificial Intelligence, Natural Language Processing, Audio Processing, and Web Development components into a single intelligent system.

3.1.1 Key Features of Python

A. Simple and Readable Syntax

One of the most important features of Python is its simple and human-readable syntax, which makes programming easier to understand and implement. Unlike many programming languages that require complex syntax and lengthy code structures, Python allows developers to write programs using fewer lines of code while maintaining clarity and readability. This simplicity helps in faster development, easier debugging, and improved maintainability. During the development of my project, Python’s readability made it easier to implement complex functionalities related to podcast transcription and Natural Language Processing.

B. Extensive Library Support

Python provides a vast collection of built-in and third-party libraries that simplify software development across multiple domains such as Artificial Intelligence, Machine Learning, Web Development, Data Science, and Audio Processing. For my project, Python libraries such as NLTK, SpaCy, SBERT, Faster Whisper, LibROSA, and PyDub played a significant role in implementing podcast transcription, topic segmentation, semantic analysis, and audio preprocessing functionalities. These pre-built libraries reduced development complexity and accelerated project implementation.

C. Platform Independence

Python is a platform-independent programming language, meaning that programs developed using Python can run on different operating systems such as Windows, Linux, and macOS without requiring major modifications. This feature ensures flexibility and portability, allowing developers to execute applications across multiple environments. During the project development process, Python's platform independence enabled smooth testing and execution of the system across different development setups.

D. Support for Multiple Programming Paradigms

Python supports multiple programming paradigms, including object-oriented programming, procedural programming, and functional programming. This flexibility enables developers to choose the most suitable programming approach according to project requirements.

E. Strong Integration Capabilities

Another important feature of Python is its ability to integrate easily with different technologies, frameworks, and APIs. Python supports seamless integration with databases, web frameworks, machine learning libraries, and external APIs, making it ideal for full-stack intelligent applications. In this project, Python was successfully integrated with Flask for backend development and multiple NLP and speech-processing libraries to build a complete podcast transcription system.

3.1.2 Use Case in My Project

During the development of the “Automated Podcast Transcription and Topic Segmentation” project, Python served as the core programming language and played a central role in implementing almost every system functionality. Since the project required integration of Artificial Intelligence, Natural Language Processing (NLP), audio processing, backend communication, and transcript analysis, Python provided a flexible and efficient environment for combining all these modules into a single intelligent system. Its rich ecosystem of libraries and simple syntax helped simplify the implementation of complex functionalities while reducing development time.

- **Audio Processing:** Python was used to preprocess podcast audio files before transcription. Libraries such as LibROSA and PyDub helped in handling audio loading, format conversion, noise reduction, and segmentation of audio files to improve overall transcription accuracy.
- **Speech-to-Text Conversion:** Python enabled the implementation of the Faster Whisper model, which was responsible for converting podcast audio into textual transcripts. The

speech recognition process transformed spoken podcast conversations into readable text while maintaining high accuracy and faster processing speed. This functionality served as the foundation of the entire system.

- **Natural Language Processing (NLP):** Python was extensively used for implementing NLP-related tasks using libraries such as NLTK, SpaCy, and SBERT. These libraries helped perform text cleaning, semantic similarity analysis, topic segmentation, keyword extraction, and transcript understanding.

- **Backend Development with Flask:** Python was integrated with the Flask framework to manage backend operations and system workflows. Flask handled functionalities such as podcast file uploads, transcript generation requests, API communication, and interaction between frontend and backend modules. This ensured smooth execution of system operations and efficient data processing.

- **Frontend Integration:** Python supported communication between backend modules and frontend technologies such as HTML, CSS, and JavaScript, allowing users to upload podcast files, access generated transcripts, and navigate topic-wise podcast sections through an interactive interface.

The implementation of Python significantly improved development efficiency and system integration by enabling smooth coordination among audio processing, transcription, Natural Language Processing, and web functionalities.

3.2 Natural Language Processing

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that focuses on enabling computers to understand, interpret, analyze, and generate human language in a meaningful way. NLP combines concepts from computer science, linguistics, machine learning, and data analysis to process textual and spoken language efficiently. It is widely used in applications such as language translation, chatbots, sentiment analysis, speech recognition, text summarization, keyword extraction, and semantic search.



Fig 3.2: Natural Language Processing

Fig 3.2 shows the concept of Natural Language Processing, which played a crucial role in the development of the “Automated Podcast Transcription and Topic Segmentation” project. Since podcasts generally consist of lengthy and unstructured spoken discussions, simply converting speech into text is often insufficient for effective understanding and navigation. Therefore, NLP techniques were implemented to analyze the generated transcript, identify meaningful topic boundaries, and organize podcast content into structured sections. This made it easier for users to quickly locate relevant discussions and understand podcast content efficiently.

To implement Natural Language Processing functionalities, multiple Python libraries such as NLTK (Natural Language Toolkit), SpaCy, and SBERT (Sentence-BERT) were integrated into the system. NLTK was used for text preprocessing tasks such as tokenization, stop-word removal, and text normalization. SpaCy helped perform linguistic analysis including sentence segmentation, named entity recognition, and syntactic processing. Meanwhile, SBERT played a major role in semantic similarity analysis by converting transcript text into vector embeddings, allowing the system to identify shifts in discussion topics and segment the podcast into meaningful sections based on contextual similarity.

The integration of NLP significantly enhanced the intelligence and usability of the system by transforming lengthy transcripts into structured, understandable, and searchable content. In addition to topic segmentation, NLP techniques also enabled keyword extraction and automatic summarization for each identified segment, helping users quickly understand the core discussion points of a podcast. Overall, Natural Language Processing served as one of the most essential technologies in the project, enabling the system to move beyond simple transcription and provide an intelligent, user-friendly podcast navigation experience.

3.2.1 Key Features of Natural Language Processing

A. Text Preprocessing and Cleaning

One of the key features of Natural Language Processing is its ability to preprocess and clean textual data before analysis. Raw text often contains unnecessary words, punctuation, symbols, and inconsistencies that can affect system performance. NLP techniques such as tokenization, stop-word removal, stemming, and lemmatization help transform

unstructured text into a structured format suitable for further processing. In this project, text preprocessing was important for improving transcript quality and preparing podcast text for topic segmentation and keyword extraction.

B. Semantic Understanding and Context Analysis

Natural Language Processing enables systems to understand the meaning and context of textual information rather than simply processing words individually. By using semantic analysis techniques, NLP can identify relationships between sentences and recognize changes in discussion topics. In this project, SBERT (Sentence-BERT) was used to perform semantic similarity analysis, helping the system detect topic transitions and divide lengthy podcast conversations into meaningful segments based on contextual understanding.

C. Text Segmentation and Topic Identification

Another important feature of NLP is the ability to segment large amounts of text into logical sections based on content similarity. This helps in identifying meaningful topic boundaries within lengthy discussions or documents. Since podcasts often contain multiple discussion points, NLP techniques were used in this project to automatically detect changes in topics and divide transcripts into organized sections. This significantly improved podcast navigation and helped users locate relevant discussions more efficiently.

D. Keyword Extraction

NLP supports automatic extraction of important keywords from textual data, allowing users to quickly understand the main focus of a conversation or document. Through keyword extraction methods, the system identifies frequently occurring and contextually significant words within podcast transcripts. In this project, keyword extraction helped summarize the major themes of each podcast segment and improved content accessibility for users.

E. Automatic Summarization

Natural Language Processing also enables automatic text summarization, which helps generate concise and meaningful summaries from lengthy content. Instead of reading entire transcripts, users can quickly understand the main discussion points through generated summaries. In the project, summarization techniques were applied to provide brief overviews of each segmented podcast section, reducing the effort required to understand long podcast discussions.

3.2.2 Application Workflow in My Project

During the development of the “Automated Podcast Transcription and Topic

Segmentation” project, Natural Language Processing (NLP) played a significant role in transforming raw podcast transcripts into meaningful, structured, and user-friendly content. Since podcasts are generally lengthy and contain multiple discussion topics, simply converting audio into text was insufficient for efficient navigation and understanding. Therefore, NLP techniques were integrated to analyze transcript content, identify topic boundaries, extract keywords, and generate concise summaries for improved accessibility and user experience.

The following are the key areas where Natural Language Processing was applied in the project:

A. Text Preprocessing and Cleaning

After converting podcast audio into text using the Faster Whisper model, the generated transcript was preprocessed to remove unnecessary symbols, punctuation, filler words, and unwanted textual noise. NLP libraries such as NLTK were used for tokenization, stop-word removal, and text normalization. This process improved transcript quality and prepared the text for accurate semantic analysis and topic segmentation.

B. Semantic Topic Segmentation

One of the primary objectives of the project was to divide lengthy podcast discussions into meaningful sections. For this purpose, SBERT (Sentence-BERT) was used to generate semantic embeddings of transcript sentences. By measuring semantic similarity between consecutive text segments, the system identified changes in discussion context and automatically detected topic boundaries. This enabled the podcast to be divided into structured and meaningful sections for easier navigation.

C. Keyword Extraction

Natural Language Processing techniques were applied to automatically identify important keywords from each segmented section of the transcript. Libraries such as SpaCy and NLTK helped analyze frequently occurring and contextually important words that represented the main focus of each discussion topic. These extracted keywords helped users quickly understand the content of different podcast sections without reading the entire transcript.

D. Automatic Summarization

To improve content accessibility, NLP-based summarization techniques were implemented to generate short summaries for each identified podcast segment. Instead of requiring users to read complete transcripts, the system provided concise overviews highlighting the main discussion points of every topic section.

E. Sentence Analysis and Linguistic Processing

Using SpaCy, advanced linguistic processing techniques such as sentence segmentation, tokenization, and contextual text understanding were applied to improve transcript organization. Sentence-level analysis helped maintain logical continuity between transcript sections and ensured accurate segmentation of podcast content based on semantic flow.

Overall, Natural Language Processing served as the intelligence layer of the project, enabling the system to move beyond basic transcription and provide meaningful topic understanding, semantic segmentation, and efficient content navigation for podcast listeners.

3.3 Faster Whisper

Faster Whisper is an advanced speech recognition model used for converting spoken audio into text efficiently and accurately. It is an optimized implementation of the Whisper Automatic Speech Recognition (ASR) model developed by OpenAI. Faster Whisper is specifically designed to improve transcription speed while maintaining high accuracy in speech recognition tasks. It supports multilingual audio processing, noise-tolerant transcription, timestamp generation, and efficient execution, making it highly suitable for applications involving audio analysis, podcast transcription, and speech-based systems. Due to its ability to process lengthy audio content quickly, Faster Whisper has become a valuable solution for speech-to-text applications.

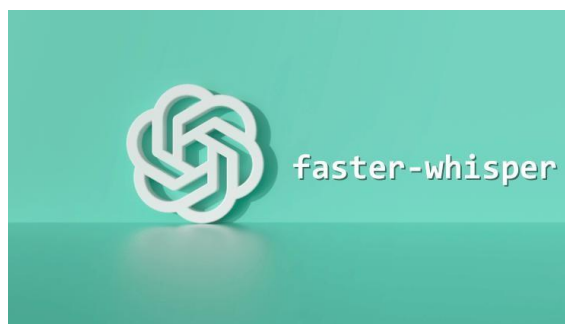


Fig 3.3: Faster Whisper

Fig 3.3 shows the Faster Whisper speech recognition model, which served as one of the most important components in the development of the “Automated Podcast Transcription and Topic Segmentation” project. Since podcasts generally consist of long-form spoken content, manually converting audio into text would be both time-consuming and inefficient. Therefore, Faster Whisper was integrated into the system to automatically convert podcast speech into accurate textual transcripts. Its ability to process speech

efficiently and handle different speaking styles significantly improved the overall functionality of the system.

During the project development process, Faster Whisper served as the primary transcription engine responsible for converting uploaded podcast audio into readable text. After the podcast audio was preprocessed using LibROSA and PyDub to improve audio quality, the cleaned audio file was passed to the Faster Whisper model for speech recognition. The generated transcript then became the input for further Natural Language Processing (NLP) tasks such as topic segmentation, keyword extraction, and automatic summarization. This integration enabled smooth communication between audio processing and transcript analysis modules.

One of the major advantages of Faster Whisper in the project was its speed and computational efficiency compared to traditional speech recognition systems. The model generated transcripts quickly while maintaining strong accuracy, even for long podcast recordings. Additionally, Faster Whisper supported timestamp-based transcription, which helped improve navigation between different podcast sections after topic segmentation. Overall, Faster Whisper acted as the backbone of the transcription system by enabling accurate, automated, and time-efficient conversion of podcast audio into structured textual content.

3.3.1 Key Features of Faster Whisper

One of the major features of Faster Whisper is its ability to perform speech-to-text conversion with high speed and efficiency. Compared to traditional transcription systems, Faster Whisper is optimized for faster processing while maintaining transcription quality. This feature was highly beneficial in the project, as podcast audio files are generally long and require efficient processing to generate transcripts within a reasonable time.

A. High Transcription Accuracy

Faster Whisper provides highly accurate speech recognition capabilities, even for lengthy audio recordings containing multiple speakers and different speaking styles. The model is designed to recognize spoken language effectively and convert it into meaningful textual output. In this project, accurate transcription was essential because the generated transcript served as the foundation for topic segmentation, keyword extraction, summarization

B. Multilingual Language Support

Another important feature of Faster Whisper is its support for multiple languages. The model can recognize and transcribe speech from different languages without requiring extensive manual configuration. This feature increases system flexibility and makes the project scalable for future improvements, where multilingual podcast transcription can be implemented for a wider audience.

C. Timestamp Generation

Faster Whisper provides timestamp-based transcription, meaning that each section of spoken content can be linked to a specific time interval in the audio file. This feature is especially useful for podcast applications, as it enables users to directly navigate to particular discussion segments within a podcast. In this project, timestamps improved topic-wise navigation and made content accessibility more convenient.

D. Noise Tolerance and Audio Adaptability

The model is capable of handling audio with minor background noise and varying sound quality. Podcasts often contain environmental disturbances, speaker interruptions, or inconsistent recording quality. Faster Whisper's adaptability to different audio conditions improved the reliability of transcript generation and reduced errors during speech recognition.

3.3.2 Application Workflow in My Project

In the "Automated Podcast Transcription and Topic Segmentation" project, Faster Whisper served as the core speech recognition engine responsible for converting podcast audio into textual transcripts. It worked seamlessly with audio preprocessing libraries, NLP modules, and the Flask backend to form an efficient end-to-end podcast analysis system. The transcription generated by Faster Whisper acted as the foundation for all subsequent processing tasks, including topic segmentation, keyword extraction, and automatic summarization.

A. Audio Input Processing

The workflow began when a user uploaded a podcast audio file through the system interface. Before transcription, the uploaded audio was prepared for processing using audio preprocessing libraries such as LibROSA and PyDub. These libraries helped load audio files, normalize sound quality, and improve clarity to ensure better speech recognition performance.

B. Speech-to-Text Conversion

After preprocessing, the cleaned podcast audio was passed to the Faster Whisper model for speech recognition. The model analyzed spoken language and converted the podcast audio into readable text automatically. This eliminated the need for manual transcription and significantly reduced the time required to process lengthy podcast discussions.

C. Transcript Generation

The generated transcript was stored temporarily and prepared for further Natural Language Processing tasks. Faster Whisper accurately converted spoken conversations into structured text while preserving sentence flow and context. This transcript became the central data source for topic segmentation and semantic analysis modules within the system.

D. Timestamp-Based Navigation

One of the key functionalities implemented in the project involved using timestamp information generated by Faster Whisper. Each segment of the transcript was associated with a specific time interval from the original podcast audio. This feature enabled users to jump directly to relevant discussion sections and improved podcast navigation efficiency.

E. Integration with Natural Language Processing

Once transcription was completed, the generated text was sent to NLP modules implemented using NLTK, SpaCy, and SBERT. These modules analyzed transcript content to identify semantic relationships, detect topic boundaries, extract keywords, and generate summaries for each identified segment. Faster Whisper therefore acted as the initial processing layer that enabled intelligent text analysis.

Overall, Faster Whisper played a central role in the project by enabling fast, accurate, and automated transcription of podcast audio, thereby supporting the intelligent analysis and structured organization of podcast content.

3.4 Audio Processing

Audio Processing plays an essential role in systems that involve speech recognition and audio analysis. It refers to the process of improving, transforming, and preparing audio signals before they are used for further computational tasks. In applications involving speech-to-text conversion, the quality of input audio directly affects transcription accuracy. Therefore, audio processing techniques are used to reduce unwanted noise, normalize sound levels, enhance speech clarity, and prepare audio files in suitable formats for machine learning and speech recognition models. In this project, audio processing served as an important preliminary step before transcription and topic analysis.

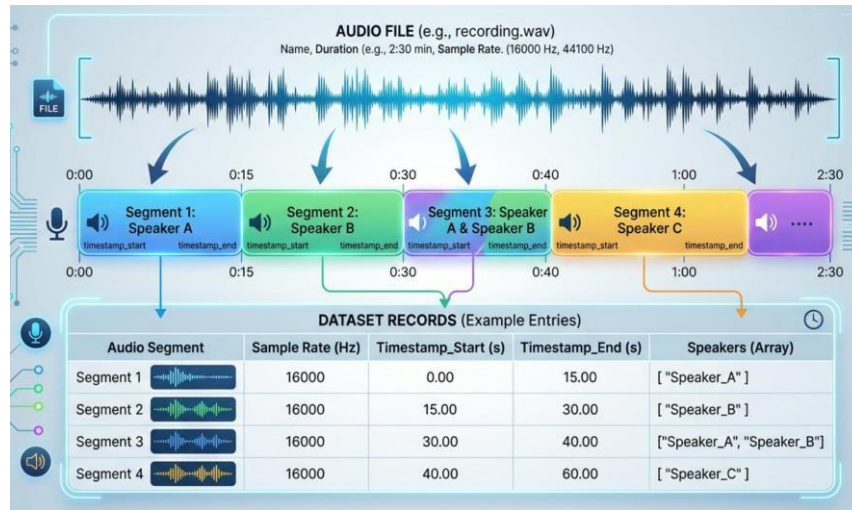


Fig 3.4: Audio Processing

Fig 3.4 shows the audio processing workflow used in the project. During the development of the “Automated Podcast Transcription and Topic Segmentation” system, audio processing was performed using Python libraries .

The audio processing stage acted as a bridge between raw podcast audio and the transcription system. Uploaded podcast files were first analyzed and cleaned to improve speech clarity and remove unnecessary distortions. Tasks such as audio normalization, format conversion, and segmentation were performed to make the audio suitable for efficient speech recognition. By enhancing audio quality before transcription, the system reduced recognition errors and improved overall transcript accuracy.

Additionally, audio processing contributed significantly to system efficiency and user experience. The integration of audio preprocessing with Faster Whisper, NLP modules, and the Flask backend helped create a smooth and intelligent workflow for converting podcast discussions into meaningful, structured, and searchable content. Overall, audio processing served as a crucial foundation for achieving reliable podcast transcription and accurate topic segmentation within the system.

Furthermore, the integration of advanced signal processing techniques allowed the system to handle diverse podcast audio conditions, including variations in background noise, speaker volume, and recording environments. By applying filtering and noise reduction methods, the system ensured that only relevant speech signals were preserved for transcription. This preprocessing step was particularly important for maintaining consistency in transcription quality across different types of podcast recordings, ranging from studio-quality audio to real-world conversational recordings. As a result, the system demonstrated robustness and adaptability in handling heterogeneous audio inputs.

In addition to improving transcription accuracy, the audio processing module played a vital role in optimizing computational performance. By segmenting long audio files into smaller, manageable chunks, the system enabled faster processing and reduced memory consumption during transcription. This chunk-based processing approach allowed the Faster Whisper model to operate more efficiently, leading to quicker response times and improved scalability. Such optimization is essential for real-world applications where users may upload lengthy podcast episodes that require timely processing and output generation.

Moreover, the audio processing stage contributed to enhancing downstream Natural Language Processing tasks, including topic segmentation, keyword extraction, and summarization. Clean and well-structured transcripts significantly improved the performance of NLP models by reducing noise and ambiguity in the input text. This, in turn, led to more accurate identification of topic boundaries and better semantic understanding of podcast content. By serving as a strong preprocessing foundation, the audio processing component ensured that the overall system delivered high-quality, meaningful, and user-friendly outputs, thereby fulfilling the objective of making podcast content more accessible and easier to navigate.

Furthermore, the audio processing module improved the overall reliability and robustness of the podcast analysis system by handling variations in audio quality, speaker pace, and background noise. Since podcast recordings may contain interruptions, varying accents, or inconsistent sound levels, preprocessing techniques such as audio normalization and segmentation helped maintain transcription consistency. This ensured that speech recognition models received optimized input, thereby minimizing transcription errors and improving the quality of generated text. As a result, users were able to obtain more precise captions, summaries, and analytical insights from uploaded podcast episodes.

Another major contribution of the audio processing pipeline was its support for scalability and system flexibility. The chunk-wise processing architecture enabled the system to handle podcasts of different durations efficiently, ranging from short interviews to lengthy multi-hour discussions. This modular design also provided opportunities for future enhancements, such as parallel audio processing, speaker diarization, multilingual transcription, and real-time streaming support.

3.4.1 Key Features of Audio Processing

A. Audio Cleaning and Noise Reduction

One of the primary features of audio processing is its ability to improve audio quality by reducing unwanted background noise and disturbances. Podcasts may contain environmental sounds, microphone disturbances, or inconsistent sound quality that can affect speech recognition accuracy. In this project, audio preprocessing techniques were applied to clean podcast recordings before transcription, helping improve speech clarity and overall transcription performance.

B. Audio Format Conversion

Podcast files are often available in different audio formats such as MP3, WAV, AAC, or FLAC. Audio processing libraries allow conversion between these formats to ensure compatibility with speech recognition systems. In this project, PyDub was used to load and convert podcast audio into suitable formats that could be processed efficiently by the Faster Whisper model for transcription.

C. Audio Normalization

Audio normalization is an important feature that adjusts sound levels to maintain consistent audio volume throughout recordings. Podcasts may contain variations in speaker volume, background intensity, or recording quality, which can negatively impact transcription accuracy. By normalizing audio signals, the system ensured balanced sound quality and improved speech recognition results.

D. Speech Clarity Enhancement

Audio processing improves the clarity of spoken content by refining audio signals before passing them to speech recognition models. Through filtering and signal enhancement techniques, important speech components become easier for the system to recognize. In this project, improving speech clarity was essential because the generated transcript served as the basis for further Natural Language Processing (NLP) tasks such as topic segmentation and summarization.

E. Audio Loading and Signal Analysis

Libraries such as LibROSA provide capabilities for loading and analyzing audio signals efficiently. They allow developers to examine waveform properties, audio duration, sample rates, and sound frequencies. In this project, LibROSA was used to process uploaded podcast audio and prepare it for efficient speech recognition by ensuring proper signal handling.

Overall, audio processing served as a critical foundation for improving audio quality,

ensuring compatibility, and increasing the effectiveness of speech recognition within the podcast transcription system.

3.4.2 Use Case in My Project

During the development of the “Automated Podcast Transcription and Topic Segmentation” project, audio processing played a crucial role in preparing podcast recordings for accurate speech recognition and analysis. Since podcast audio files often contain background noise, varying sound quality, and long-duration recordings, preprocessing the audio before transcription was essential to ensure reliable system performance and better transcription accuracy.

The following are the major ways audio processing was used in the project:

- **Audio Preprocessing Before Transcription**

Before converting speech into text, uploaded podcast audio files were first processed using LibROSA and PyDub. These libraries helped clean and prepare the audio by handling sound inconsistencies and improving quality. This preprocessing step ensured that the Faster Whisper speech recognition model received optimized audio input for accurate transcription.

- **Noise Reduction and Quality Improvement**

Podcast recordings may include background disturbances, microphone noise, or inconsistent audio quality. Audio processing techniques were used to reduce unwanted noise and improve speech clarity, making it easier for the transcription model to recognize spoken words correctly. This significantly reduced transcription errors and improved transcript quality.

- **Audio Format Conversion**

Since podcast files can be uploaded in multiple formats such as MP3, WAV, or AAC, audio processing tools were used to convert audio into compatible formats suitable for transcription. This ensured flexibility within the system and allowed users to upload podcasts in different file types without compatibility issues.

- **Handling Long Podcast Recordings**

Many podcast episodes are lengthy and require efficient processing. Audio processing techniques helped manage long recordings by analyzing and preparing audio streams effectively before transcription. This ensured smooth system performance and prevented processing delays when handling large podcast files.

- **Integration with Faster Whisper**

Audio processing served as a bridge between uploaded podcast audio and the Faster

Whisper transcription system. Once preprocessing was completed, the cleaned audio was passed to Faster Whisper for speech-to-text conversion. This integration helped generate more accurate transcripts, which later supported topic segmentation and keyword extraction.

- **Supporting NLP-Based Analysis**

The quality of transcript output directly influenced the effectiveness of Natural Language Processing (NLP) tasks such as semantic topic segmentation, keyword extraction, and summarization. By improving audio quality before transcription, audio processing indirectly enhanced the accuracy of these advanced text analysis features.

- **Improving Overall User Experience**

Through proper audio optimization and preprocessing, the system provided users with accurate transcripts and meaningful podcast segmentation results. This improved podcast accessibility, reduced manual effort, and made it easier for users to navigate and understand lengthy podcast discussions efficiently.

Overall, audio processing played a foundational role in ensuring smooth workflow execution, improving transcription accuracy, and enhancing the performance of the entire podcast transcription and topic segmentation system.

3.5 Flask

Flask is a lightweight and open-source Python web framework used for developing web applications and APIs quickly and efficiently. It is designed to be simple, flexible, and easy to integrate with various libraries and technologies. Unlike large frameworks, Flask follows a minimalistic approach, allowing developers to add functionalities as needed. Due to its simplicity and scalability, Flask is widely used in Artificial Intelligence, Machine Learning, and data-driven applications where backend communication and API integration are required.

The Flask framework, which played an important role in the development of the “Automated Podcast Transcription and Topic Segmentation” project. Since the system required communication between the frontend interface and backend processing modules, Flask was used as the primary backend framework to handle requests, process podcast audio, and return transcription and segmentation results. Its lightweight architecture made it suitable for integrating Artificial Intelligence models and Natural Language Processing functionalities efficiently.

During the project development process, Flask acted as the bridge between user interaction and backend processing workflows. When users uploaded podcast audio files

through the interface, Flask handled file management and routed requests to various processing modules such as audio preprocessing, Faster Whisper transcription, NLP-based topic segmentation, keyword extraction, and summarization. After processing was completed, Flask returned the generated transcripts and topic-wise outputs back to the frontend for display.

One of the major advantages of Flask in this project was its flexibility and seamless integration with Python-based libraries such as NLTK, SpaCy, SBERT, Faster Whisper, LibROSA, and PyDub. Since all core project modules were implemented in Python, Flask enabled smooth communication between these components without compatibility issues. Additionally, Flask helped simplify API development, request handling, and backend management, ensuring efficient system performance.

Overall, Flask served as the backbone of the backend architecture by enabling smooth interaction between audio processing, transcription generation, and Natural Language Processing modules. Its lightweight design, scalability, and ease of integration significantly contributed to the successful implementation of the podcast transcription and topic segmentation system.

3.5.1 Key Features of Flask

A. Lightweight and Minimal Framework

One of the major features of Flask is its lightweight and minimalistic architecture. Flask provides only the essential components required for web development, allowing developers to add extra functionalities based on project requirements. Unlike larger frameworks, Flask avoids unnecessary complexity and offers greater flexibility in application design. In this project, Flask provided a simple and efficient backend framework for handling podcast processing workflows.

B. Easy API Development

Flask makes it easy to develop and manage RESTful APIs, which are essential for communication between frontend and backend systems. Developers can quickly create routes and endpoints to process requests and return responses. In the podcast transcription project, Flask APIs handled operations such as podcast file uploads, transcript generation requests, topic segmentation, keyword extraction, and result delivery to the frontend interface.

C. Seamless Integration with Python Libraries

Since Flask is built using Python, it integrates smoothly with various Python-based libraries and frameworks. This makes it highly suitable for Artificial Intelligence and Machine

Learning applications. In this project, Flask worked efficiently with libraries such as Faster Whisper, NLTK, SpaCy, SBERT, LibROSA, and PyDub, enabling smooth communication between transcription, NLP, and audio processing modules.

3.5.2 Application Workflow

A. Flask Backend Architecture and Setup

During the development of the Automated Podcast Analysis System, Flask was used as the core backend framework to handle API requests, process podcast data, and communicate with the frontend interface.

The application was structured using a modular approach, separating routes, services, and utility functions. Flask's lightweight nature allowed quick development of RESTful APIs that handled tasks such as audio upload, transcription processing, and retrieval of analyzed podcast insights.

Environment configuration and dependency management were handled using virtual environments to ensure consistency across development and deployment setups.

B. API Development and Request Handling

Flask routes were created to manage different functionalities of the system, such as uploading podcast files, fetching transcript data, and generating summaries.

Each API endpoint was designed to accept JSON or file inputs, process them through backend logic, and return structured JSON responses. This ensured smooth integration with the frontend and chatbot modules.

Proper request validation and error handling mechanisms were implemented to manage invalid inputs, missing files, and processing failures, ensuring system stability.

C. Integration with AI/ML Processing Pipeline

The Flask backend acted as a bridge between the user interface and the AI processing modules used for podcast analysis.

Once a podcast was uploaded, Flask triggered the processing pipeline, which included transcription generation, keyword extraction, topic detection, and sentiment analysis. The processed results were then stored and served back through API endpoints.

This integration ensured that heavy computation tasks were handled efficiently without blocking the main application flow.

D. Data Management and Response Structuring

Flask was also responsible for organizing and serving processed podcast data in a structured format. The results were formatted into JSON objects containing transcript segments, summaries, topics, and timestamps.

This structured response format made it easier for the frontend dashboard and chatbot interface to display insights in a user-friendly manner.

Caching strategies were also considered to reduce repeated processing of the same podcast data, improving system performance and response time.

3.5.3 Learning Outcomes and Benefits

Working on the Automated Podcast Analysis System using Flask and Git-based development significantly enhanced my technical and problem-solving skills in full-stack and backend development. Some key takeaways include:

- Gained strong understanding of designing and developing RESTful APIs using Flask for real-world applications such as podcast uploading, transcription, and analysis retrieval.
- Improved knowledge of backend architecture design, including modular structuring of routes, services, and utility components for better scalability and maintainability.
- Learned effective integration of AI/ML pipelines with backend systems, enabling seamless processing of podcast data for transcription, summarization, and sentiment analysis.

Chapter 4

TECHNICAL TRAINING AND ASSIGNMENT

The initial phase of my internship through the Infosys Springboard virtual internship program focused on technical learning, practical implementation, and project development in the domain of Artificial Intelligence and software development. This training phase was designed to strengthen both theoretical understanding and practical skills by introducing modern technologies required for developing intelligent systems. Since my project involved podcast transcription and topic segmentation, the training mainly emphasized concepts related to Python programming, Natural Language Processing (NLP), speech recognition, audio processing, backend development, and frontend integration. The primary objective of this phase was to build a strong technical foundation before moving toward actual project implementation.

The training process followed a practical and structured learning approach where I explored technologies through hands-on experimentation rather than only theoretical study. Initially, I strengthened my understanding of Python, which served as the core programming language for the entire project. I learned how to work with Python libraries, file handling, API integration, and backend functionalities. Alongside Python, I gained knowledge of Natural Language Processing techniques, including text preprocessing, semantic similarity analysis, keyword extraction, and summarization using libraries such as NLTK, SpaCy, and SBERT. Since the project involved speech-based content, I also explored speech-to-text technologies, specifically Faster Whisper, to understand how audio could be transformed into accurate textual transcripts.

Another major part of the training involved understanding audio processing and system integration. I learned to work with libraries such as LibROSA and PyDub for audio cleaning, normalization, and preprocessing to improve speech recognition performance. Additionally, I gained practical exposure to backend development using Flask, which was used to create APIs, manage requests, and connect different processing modules within the application. For frontend interaction, I explored web technologies such as HTML, CSS, and JavaScript to create a simple and user-friendly interface that allowed podcast upload, transcript viewing, and topic-based navigation.



Fig 4.1: Training Program Flow

The most important outcome of this training phase was the successful implementation of the project titled “Automated Podcast Transcription and Topic Segmentation.” This project allowed me to apply the concepts learned during training in a practical environment. I worked on designing the system workflow, integrating multiple technologies, processing podcast audio, generating transcripts, identifying topic boundaries, extracting keywords, and creating summaries for segmented content. The project also helped me understand debugging, testing, backend integration, and efficient system design practices. Overall, the training phase provided me with valuable technical exposure, improved my programming and analytical skills, and prepared me to work on Artificial Intelligence-based real-world applications confidently.

4.1 Python Programming and AI/ML Fundamentals

Python served as the fundamental programming language throughout my internship project due to its simplicity, flexibility, and strong support for Artificial Intelligence and Machine Learning applications. Since the project “Automated Podcast Transcription and Topic Segmentation” required handling audio processing, Natural Language Processing (NLP), speech recognition, and backend development, gaining a strong understanding of Python became an essential part of the technical training process. Python’s readable syntax and extensive ecosystem of libraries made it highly suitable for developing intelligent systems capable of processing and analyzing large amounts of audio and text data efficiently.

The learning process began with understanding the core fundamentals of Python, including variables, data types, operators, loops, conditional statements, functions, object-oriented programming concepts, exception handling, file handling, and module creation.

A gradual and structured approach was followed to strengthen programming logic and problem-solving skills. Along with theoretical understanding, practical implementation through coding exercises and mini-tasks helped build confidence in writing efficient and reusable code. This foundational knowledge proved extremely useful while handling complex project modules and integrating multiple technologies.

As the training progressed, the focus shifted toward Artificial Intelligence and Machine Learning fundamentals, particularly concepts relevant to audio and text analysis. I gained practical exposure to AI-based workflows, including data preprocessing, text analysis, semantic similarity detection, speech-to-text conversion, and automated summarization. Libraries such as NLTK, SpaCy, SBERT, Faster Whisper, LibROSA, and PyDub were explored to understand how AI models process spoken language and generate meaningful insights from raw podcast audio. This exposure helped me understand the importance of data cleaning, feature extraction, model integration, and optimization in intelligent systems.

Furthermore, Python played a major role in integrating various project modules into a single functional application. It enabled smooth implementation of audio preprocessing, transcript generation, topic segmentation, keyword extraction, and backend API handling using Flask. Working with Python throughout the internship significantly improved my logical thinking, debugging abilities, and coding efficiency. Overall, learning Python and AI/ML fundamentals laid a strong technical foundation for successfully developing the podcast analysis system and strengthened my understanding of real-world Artificial Intelligence applications.

4.1.1 Core Python Concepts for Backend Development

At the foundational level, I learned the core concepts of Python programming, which formed the basis for backend development in my project. The learning process started with understanding variables, data types, and memory handling in Python. I explored different data types such as integers, floating-point numbers, strings, lists, tuples, dictionaries, sets, and Boolean values. Understanding how variables are declared and manipulated in Python helped me write structured and efficient code for managing podcast data and system workflows.

The training also focused on operators and conditional statements, which are essential for implementing logic in software applications. I practiced arithmetic, relational, logical, assignment, and membership operators to build condition-based functionality. Control structures such as if-else statements, nested conditions, loops (for and while),

break, continue, and pass statements were extensively used through programming exercises. These concepts helped me implement decision-making processes in the project, such as handling user requests, validating uploaded podcast files, and managing transcript processing conditions.

The training further covered file handling and exception handling, which are critical for backend systems. I learned how to read and write files, handle uploaded podcast audio files, and manage text transcripts efficiently. Exception handling using try, except, finally, and custom exceptions helped improve system reliability by preventing unexpected failures during audio processing and API execution. This was especially important in the project to ensure smooth handling of errors related to file uploads, unsupported formats, or processing interruptions.

In addition, I explored Object-Oriented Programming (OOP) concepts such as classes, objects, inheritance, polymorphism, encapsulation, and abstraction. These concepts helped me understand how to organize code efficiently and build scalable systems. Backend development also required understanding Python libraries and package management, where I learned to integrate external libraries such as Flask, Faster Whisper, NLTK, SpaCy, LibROSA, and PyDub into the project workflow. Overall, mastering these core Python concepts established a strong programming foundation and enabled me to effectively build and integrate backend functionalities for the podcast analysis system.

4.1.2 Data Processing and Automation Concept

After building a strong foundation in Python programming, the next stage of training focused on data processing and automation concepts, which played a major role in the development of the Automated Podcast Transcription and Topic Segmentation system. Since the project involved handling large audio files, converting speech into text, and analyzing transcript data, understanding how to automate repetitive tasks and efficiently process data became extremely important. This phase helped me understand how intelligent systems manage data flow, transform raw inputs, and generate meaningful outputs automatically.

Initially, I learned about data handling and preprocessing techniques in Python. This included working with different file formats, reading and processing text data, manipulating strings, handling audio files, and managing structured data using lists, dictionaries, and data frames. I explored how raw podcast audio needed to be cleaned and prepared before analysis. Libraries such as LibROSA and PyDub were used for tasks like audio normalization, silence removal, noise reduction, and format conversion to improve

the quality of speech recognition.

The training also emphasized automation concepts, where Python scripts were used to reduce manual effort and streamline repetitive processes. By integrating multiple modules together, the system was able to automatically process uploaded podcasts and generate structured results without human intervention. This automation significantly improved efficiency and reduced processing time.

Another important learning area was data processing using Natural Language Processing (NLP) techniques. Once the podcast audio was converted into text through Faster Whisper, the transcript underwent preprocessing steps such as tokenization, stop-word removal, text cleaning, and semantic analysis. NLP libraries like NLTK, SpaCy, and SBERT were used to identify topic boundaries, understand contextual similarity between sentences, and divide lengthy podcast transcripts into meaningful sections. This enabled the system to generate organized content for easier navigation and understanding.

Furthermore, I learned the importance of error handling, workflow management, and backend automation while integrating different project modules using Flask APIs. Techniques such as exception handling, logging, and process validation ensured smooth system performance even when unexpected issues occurred during transcription or audio analysis. Overall, learning data processing and automation concepts significantly improved my ability to build intelligent workflows, manage complex datasets, and develop efficient AI-driven systems capable of processing podcast content automatically.

4.1.3 AI/ML Concepts for Text Analysis

An important phase of the training focused on understanding Artificial Intelligence (AI) and Machine Learning (ML) concepts related to text analysis, which played a significant role in the development of the Automated Podcast Transcription and Topic Segmentation project. Since the project required transforming raw podcast conversations into meaningful and structured information, it became essential to understand how machines process, interpret, and analyze human language. This learning phase introduced me to the practical application of AI and NLP techniques for extracting meaningful insights from unstructured text data.

Initially, I learned the fundamentals of Natural Language Processing (NLP), which is a branch of Artificial Intelligence that enables computers to understand, interpret, and generate human language. I explored important preprocessing techniques such as text cleaning, tokenization, stop-word removal, stemming, lemmatization, and sentence segmentation. These preprocessing steps were crucial because raw transcripts generated

from podcast audio often contain unnecessary words, repetitions, and noise that need to be refined before meaningful analysis can take place.

training further focused on semantic text analysis and contextual understanding, where I explored how AI models identify relationships between sentences and topics within large textual data. Libraries such as SpaCy and SBERT (Sentence-BERT) were introduced to analyze sentence embeddings and semantic similarity. These techniques enabled the system to understand context and identify transitions between different podcast discussions, which ultimately helped in automatic topic segmentation. This process ensured that lengthy podcast transcripts could be divided into meaningful topic-based sections for easier understanding and navigation.

Furthermore, I gained practical exposure to machine learning-driven text representation methods, where textual information was converted into numerical embeddings for efficient processing and comparison. These concepts helped me understand how intelligent systems recognize patterns in text and generate accurate results. During project implementation, these AI/ML concepts were integrated to improve transcript organization, contextual understanding, and user accessibility. Overall, this phase strengthened my understanding of intelligent text analysis systems and provided valuable practical experience in applying Artificial Intelligence for solving real-world content accessibility challenges.

4.2 Backend Development with Flask

Backend development plays a crucial role in managing the core functionality, logic, and communication processes of any software application. During my internship training, I gained practical exposure to backend development using Flask, a lightweight and flexible Python web framework widely used for building APIs and web applications. Since my project, “Automated Podcast Transcription and Topic Segmentation,” involved integrating multiple Artificial Intelligence and audio processing modules, Flask served as the backbone of the entire system by enabling smooth communication between the frontend interface and backend processing components.

Flask provided a simple yet powerful framework for creating APIs, handling user requests, processing podcast uploads, and delivering analyzed results efficiently. Through practical implementation, I learned how backend systems manage application workflows, process requests, and coordinate interactions between different technologies. Flask’s lightweight nature made it highly suitable for integrating advanced AI libraries such as Faster Whisper, NLTK, SpaCy, SBERT, LibROSA, and PyDub, enabling smooth

execution of audio transcription, topic segmentation, keyword extraction, and summarization tasks within a single platform.

During the training and development phase, I explored the complete backend development lifecycle using Flask—from creating a basic server to building APIs for real-world functionalities. I learned how to define routes, handle HTTP requests such as GET and POST, manage uploaded files, process data dynamically, and return structured responses to users. Flask also helped in connecting different project modules together, ensuring that podcast audio moved seamlessly through stages such as preprocessing, speech-to-text conversion, transcript analysis, and final output generation.

Overall, learning backend development with Flask provided me with valuable hands-on experience in designing and implementing intelligent web-based systems. It strengthened my understanding of API development, system integration, server-side logic, and backend workflow management. This knowledge played an essential role in the successful development of the podcast transcription and topic segmentation system, allowing different technologies to work together in an efficient and organized manner.

4.2.1 Setting Up the Development Environment and Flask Basics

My learning journey in backend development began with setting up the Python and Flask development environment, which served as the foundation for building the backend system of the Automated Podcast Transcription and Topic Segmentation project. Initially, I learned how to install and configure Python packages using pip (Python Package Installer) and create isolated environments using virtual environments (venv) to manage dependencies efficiently. Understanding package management helped me organize libraries and maintain compatibility between different project modules without affecting the system environment.

During the setup phase, I explored the basic structure and working of the Flask framework. I learned how to initialize Flask applications, configure routes, and run a local development server for testing backend functionalities. Flask's simple syntax made it easier to understand the fundamentals of backend development. I experimented with creating basic APIs and learned how web applications process requests and generate responses through HTTP methods such as GET and POST. These concepts formed the foundation for implementing podcast upload and transcript processing functionalities later in the project.

To strengthen my understanding, I also explored how Flask handles request-response workflows, file uploads, and API communication. Since the project required users

to upload podcast audio files for analysis, I practiced handling file management operations such as receiving uploaded files, validating file types, storing temporary data, and returning structured responses. Flask enabled smooth interaction between the frontend interface and backend modules responsible for audio preprocessing and transcription.

To reinforce backend concepts, I performed practical tasks involving file processing, API handling, and automation workflows. I implemented routes for receiving podcast audio, triggering transcription generation using Faster Whisper, and passing transcript data to NLP modules for topic segmentation and keyword extraction. This hands-on experience helped me understand how different technologies communicate within a backend system and strengthened my ability to develop scalable, efficient, and intelligent applications using Flask.

4.2.2 RESTful API Development for Podcast Processing

After gaining a strong understanding of Flask fundamentals, the next phase focused on building RESTful APIs for the Automated Podcast Transcription and Topic Segmentation project. RESTful APIs played an essential role in enabling communication between the frontend interface and backend processing modules. Since the application required functionalities such as podcast file uploading, transcript generation, topic segmentation, keyword extraction, and result retrieval, APIs acted as the bridge that connected these different components into a unified system.

Using Flask, I learned how to create structured API routes using HTTP methods such as GET, POST, PUT, and DELETE. The primary focus was on handling podcast-related operations efficiently. For example, POST requests were used for uploading podcast audio files and initiating transcript processing, while GET requests allowed retrieval of generated transcripts, segmented topics, extracted keywords, and summarized content. Understanding these request-response mechanisms helped me design an organized and scalable backend architecture.

Each API endpoint was developed carefully to manage file handling, request validation, and response generation. Flask routes were configured to process uploaded audio files, validate supported formats, and pass the podcast data through multiple processing stages such as audio preprocessing, speech-to-text conversion using Faster Whisper, and NLP-based topic segmentation. Middleware-like validation logic was also implemented to ensure proper request handling and avoid invalid inputs or unsupported file types.

A major improvement in backend organization came from following a modular

architecture, where the application was divided into separate processing modules for transcription, segmentation, keyword extraction, and summarization. This structured design made the codebase easier to maintain, debug, and scale for future enhancements. It also simplified integration between Flask APIs and frontend components, ensuring smooth user interaction and efficient data flow throughout the application.

4.2.3 Integration of AI Processing Pipeline

One of the most important aspects of my internship project was the integration of an Artificial Intelligence (AI)-based processing pipeline for podcast analysis. Since the project focused on “Automated Podcast Transcription and Topic Segmentation,” it required combining multiple AI and Natural Language Processing (NLP) techniques into a single workflow. The backend system was designed to process podcast audio, convert speech into text, identify topic changes, extract keywords, and generate meaningful summaries. This integration helped transform lengthy podcast audio into organized and user-friendly content.

The AI processing pipeline was implemented using Python and integrated with the backend using Flask APIs. Each stage of the system was designed to perform a specific task while maintaining a smooth flow of data between modules. The complete workflow started from audio upload and progressed through preprocessing, transcription, topic segmentation, semantic analysis, and summarization. This modular architecture made the application scalable and easier to maintain.

To improve system performance and audio quality before transcription, audio preprocessing techniques were applied using libraries such as LibROSA and PyDub. These tools helped in handling audio normalization, silence trimming, format conversion, and noise reduction. Clean audio input significantly improved the accuracy of speech recognition and reduced transcription errors in noisy podcast recordings.

For speech-to-text conversion, I implemented the FasterWhisper model, which provided efficient and high-quality transcription results. FasterWhisper was chosen due to its fast inference speed and better optimization for long audio files. The model converted spoken podcast content into textual transcripts while preserving timestamps, which later helped in identifying topic boundaries within the podcast.

After transcription, Natural Language Processing (NLP) techniques were applied to analyze the generated text. Libraries such as NLTK, SpaCy, and SBERT (Sentence-BERT) were used to process the transcript and understand semantic relationships between sentences.

4.2.3.1 Transcript Processing and Text Cleaning

The generated transcript was first cleaned and preprocessed before performing advanced analysis. This stage involved:

- Removing unnecessary stop words and special characters.

- Sentence tokenization and text normalization.

- Lemmatization to reduce words to their root forms.

- Filtering irrelevant textual noise for better semantic understanding.

This preprocessing step improved the quality of downstream NLP operations.

4.2.3.2 Semantic Topic Segmentation

A core functionality of the project was automatically dividing long podcast transcripts into meaningful topic-based sections. To achieve this:

- Sentence embeddings were generated using SBERT.

- Semantic similarity between consecutive transcript segments was calculated.

- Topic boundaries were identified whenever significant semantic shifts occurred.

- Related discussions were grouped into logical podcast sections.

This feature enabled users to navigate podcasts more efficiently without listening to the entire recording.

4.2.3.3 Keyword Extraction and Content Analysis

To enhance podcast accessibility, keyword extraction techniques were applied to identify important terms and concepts discussed in each segment. This involved:

- Identifying frequently occurring meaningful words.

- Extracting topic-specific phrases.

- Generating metadata for improved podcast searchability.

- Highlighting major discussion areas within the podcast.

This functionality made it easier for users to quickly understand the focus of each section.

4.2.3.4 Automated Summarization

The final stage of the AI pipeline involved generating summaries for each identified topic segment. NLP-based summarization techniques were used to:

- Create concise descriptions of long discussions.

- Highlight important points from podcast conversations.

- Reduce the effort required to understand lengthy content.

- Improve overall user experience through structured insights.

Integrating all these AI modules into a unified processing pipeline significantly improved the effectiveness of the project. It allowed seamless automation of podcast analysis while

ensuring that users received structured, meaningful, and time-efficient access to podcast content. This experience gave me practical exposure to integrating AI, NLP, backend development, and audio processing into a real-world application.

4.2.4 Handling Data Flow and Backend Logic

Efficient data flow and backend logic management were essential components of my internship project, as the system needed to handle podcast uploads, audio processing, AI-based transcription, topic segmentation, and result generation in a smooth and organized manner. Since the project involved multiple interconnected modules, it was important to design a backend architecture capable of efficiently transferring data between different stages of processing. Using Flask, I developed APIs that managed requests, processed podcast files, interacted with AI models, and returned structured outputs to the frontend.

The backend system followed a sequential workflow where each module performed a dedicated responsibility. Once a podcast audio file was uploaded by the user, the backend initiated preprocessing operations before sending the cleaned audio for transcription using the FasterWhisper model. The generated transcript was then passed to Natural Language Processing (NLP) modules for topic segmentation, keyword extraction, and summarization. The final processed results were sent back to the frontend interface for visualization and interaction.

To maintain efficient communication between different modules, Flask APIs were designed to handle data requests and responses in a structured format using JSON (JavaScript Object Notation). This enabled seamless transfer of transcription results, segmented topics, summaries, and extracted keywords between the backend and frontend system.

A. Podcast Upload and Request Handling

The first stage of backend logic involved handling podcast uploads through Flask APIs. The system was designed to:

- Accept podcast audio files from users.
- Validate file formats and size restrictions.
- Store uploaded files temporarily for processing.
- Trigger backend workflows automatically after upload.

This ensured smooth interaction between the user interface and the processing system.

B. Audio Processing Workflow Management

Once the file was uploaded, backend logic controlled the complete audio processing sequence:

- Audio cleaning and normalization using PyDub and LibROSA.
- Format conversion to ensure compatibility with transcription models.
- Silence removal and preprocessing to improve transcription accuracy.
- Passing processed audio to the transcription engine.
- Managing this flow properly helped maintain consistency and reduced processing errors.

C. AI Model Integration and Data Flow

The backend coordinated communication between multiple AI and NLP modules. This involved:

- Sending processed audio to the FasterWhisper model for speech-to-text conversion.
- Passing generated transcripts to SpaCy, NLTK, and SBERT for text analysis.
- Managing semantic similarity calculations for topic segmentation.
- Returning extracted keywords and summaries for each topic section.
- This integration ensured that outputs from one module became inputs for the next stage without interruption.

D. API Response Handling and Result Delivery

After completing analysis, Flask APIs generated structured responses that were returned to the frontend application. These responses included:

- Complete podcast transcripts.
- Topic-wise segmented sections.
- Automatically generated summaries.
- Important keywords and metadata.
- Using JSON responses made frontend integration easier and allowed dynamic rendering of results.

Overall, handling backend logic and data flow taught me how to manage complex workflows involving AI models, APIs, and multiple processing stages within a single application.

4.3 Frontend Integration and User Interface

Frontend integration and user interface design played a significant role in the successful implementation of my internship project. Since the project involved converting lengthy podcast audio into structured and understandable information, it was important to build a user-friendly interface that allowed users to upload podcast files, view transcriptions,

explore segmented topics, and access summaries conveniently. The frontend acted as the communication bridge between users and the AI-powered backend system, ensuring smooth interaction and accessibility.

For frontend development, I used HTML, CSS, JavaScript, and Bootstrap to design a responsive and interactive user interface. The interface was designed with simplicity and usability in mind so that users could easily upload podcast audio files and navigate through processed results without technical complexity. Proper frontend-backend communication was established using Flask APIs, enabling real-time exchange of data between the interface and backend processing modules.

The frontend was carefully integrated with the backend system to handle different functionalities such as file uploads, transcript display, topic segmentation visualization, and result presentation. The application workflow ensured that users could interact with the system smoothly while the backend handled intensive AI processing tasks in the background.

A. Podcast Upload Interface

One of the primary features of the user interface was the podcast upload system. This module was designed to:

- Allow users to upload podcast audio files easily.
- Support multiple audio formats for compatibility.
- Validate uploaded files before processing.
- Provide a simple and intuitive interaction flow.
- This feature served as the entry point for the entire podcast analysis process.

B. Frontend and Backend Communication

The frontend was integrated with the Flask backend using API requests to enable smooth data transfer. This communication involved:

- Sending uploaded podcast files to Flask endpoints.
- Receiving transcription and segmentation results dynamically.
- Fetching processed summaries and keywords.
- Updating interface components based on backend responses.
- This ensured real-time interaction between the interface and AI processing modules.

C. Displaying Transcription Results

After podcast processing was completed, the frontend displayed generated transcripts in an

organized manner. This functionality included:

- Showing complete podcast transcripts in readable text format.
- Preserving timestamps for better navigation.
- Structuring content for easier readability.
- Allowing users to review processed audio content efficiently.
- This made long podcast discussions easier to understand and revisit.

D. Topic Segmentation Visualization

A major feature of the user interface was displaying topic-based segmented sections generated by NLP models. The interface helped users:

- Identify different discussion topics within podcasts.
- Navigate directly to specific sections of interest.
- Understand topic transitions clearly.
- Improve accessibility of long-form audio content.
- This significantly enhanced user experience by reducing the need to listen to the entire podcast.

Overall, frontend integration and user interface development played an important role in making the project practical, interactive, and user-friendly. It helped transform complex AI-generated outputs into visually understandable information for end users. This experience improved my understanding of frontend-backend integration, API communication, responsive web design, and creating user-centric software applications.

4.3.1 Connecting Flask Backend with Frontend

An important aspect of my internship project was establishing smooth communication between the frontend interface and the Flask backend system. Since the project involved podcast uploading, audio processing, transcription, topic segmentation, and result generation, efficient integration between frontend and backend was essential to ensure seamless functionality. The frontend acted as the user interaction layer, while Flask managed the backend processing logic and communication with Artificial Intelligence models.

The integration process involved creating Flask API endpoints that allowed the frontend to send requests and receive processed responses dynamically. These APIs acted as a bridge between the user interface and backend functionalities, enabling users to upload podcast files and retrieve generated transcripts, summaries, segmented topics, and

extracted keywords. Proper API communication ensured smooth data flow and enhanced overall application performance.

To establish connectivity between the frontend and Flask backend, I implemented HTTP request methods such as GET and POST for handling data exchange. This allowed frontend components to send user input and fetch processed information efficiently.

A. Handling Podcast Upload Requests

One of the first integration tasks involved connecting the podcast upload interface with the Flask backend. This process included:

- Allowing users to upload podcast audio files through the frontend.
- Sending uploaded files to Flask APIs using POST requests.
- Validating audio formats before processing.
- Triggering backend processing automatically after file submission.
- This ensured a smooth and user-friendly upload experience.

B. API-Based Communication

To facilitate communication between frontend and backend modules, Flask APIs were developed for different operations such as:

- Audio upload and preprocessing.
- Speech-to-text transcription generation.
- Topic segmentation and semantic analysis.
- Keyword extraction and automated summarization.
- Each API endpoint handled a specific task and returned structured data responses in JSON format.

C. Sending and Receiving Data Using JSON

JSON played an important role in frontend-backend communication because it allowed structured data transfer between application components. The frontend:

- Sent podcast file information and user requests to Flask APIs.
- Received transcript results, segmented topics, and summaries.
- Dynamically rendered returned information on the interface.
- Updated UI components without reloading the entire page.

This improved responsiveness and overall user experience.

Overall, connecting the Flask backend with the frontend was a crucial step in transforming the project into a fully functional web application. It enabled efficient communication between users and AI-powered backend services, ensuring that podcast processing results

were delivered in an organized, responsive, and user-friendly manner. This experience strengthened my understanding of API integration, backend communication, dynamic rendering, and full-stack application development.

4.3.2 Real-World Application and Dashboard

An important objective of the internship project was to develop a practical and user-friendly system capable of solving real-world challenges associated with podcast consumption. Since podcasts are often lengthy and unstructured, users may find it difficult to identify important discussions or revisit specific topics efficiently. To address this issue, the project was designed as an intelligent application that automatically converts podcast audio into structured, searchable, and topic-wise organized content. The system provided users with a centralized dashboard where they could upload podcast files, access transcripts, view segmented topics, and analyze summarized content conveniently.

The dashboard served as the primary interaction interface between users and the AI-powered backend system. It was designed to simplify user experience while presenting podcast insights in a structured and understandable manner. By integrating frontend technologies with Flask APIs, the dashboard dynamically displayed outputs generated through transcription and Natural Language Processing (NLP) modules. This seamless integration ensured smooth communication between the frontend and backend, enabling users to interact with the system without technical complexity.

The application workflow was carefully designed to provide a smooth user experience from the initial podcast upload stage to the final display of analyzed content. Users could upload podcast audio files directly into the system, where the backend automatically validated file formats and initiated audio preprocessing and transcription. The FasterWhisper model generated accurate transcripts, which were then processed through NLP techniques for topic segmentation, keyword extraction, and summarization. The processed information was finally displayed on the dashboard in a clear and user-friendly format.

One of the major functionalities of the dashboard was transcript visualization and navigation. Instead of manually listening to long podcast recordings repeatedly, users could read generated transcripts in an organized format. Timestamp-based segmentation allowed users to quickly navigate to specific portions of the discussion, making podcast content more accessible and time-efficient. This significantly improved content consumption and reduced the effort required to revisit important discussions.

Overall, the developed application addressed a genuine problem faced by podcast

listeners by making lengthy audio content easier to consume, navigate, and analyze. The system can be beneficial for students reviewing educational podcasts, professionals extracting valuable insights from discussions, content creators organizing podcast archives, and general users revisiting important topics efficiently. Developing this real-world application enhanced my understanding of Artificial Intelligence, frontend-backend integration, API communication, and user-centric software development, while strengthening my ability to build intelligent and scalable systems.

4.4 Version Control with Git and GitHub

Version control is an essential practice in software development, especially when working on projects that involve multiple updates, experimentation, and continuous improvement. During my internship, I used Git and GitHub extensively to manage source code, track project progress, and maintain organized development workflows for the Automated Podcast Transcription and Topic Segmentation project. These tools helped ensure that every modification made to the project could be monitored, reverted if necessary, and collaboratively maintained in an efficient manner.

Git served as the primary version control system for tracking changes in both the frontend and backend components of the application. Since the project involved integrating multiple modules such as audio preprocessing, FasterWhisper transcription, Natural Language Processing (NLP), Flask API development, and frontend dashboard integration, maintaining clean and structured code became highly important. By using Git, I was able to create checkpoints of the project through commits, making it easier to manage updates and avoid accidental code loss.

GitHub acted as a cloud-based repository platform where the project code was securely stored and organized. It enabled centralized access to the source code and simplified project management by maintaining a complete history of changes. Throughout the development process, I regularly pushed updated code to GitHub after implementing new features such as podcast upload functionality, transcript generation, topic segmentation modules, keyword extraction, and dashboard enhancements.

The use of Git and GitHub during the internship improved my understanding of professional software development practices. I learned how to maintain clean commit histories, organize code effectively, handle version conflicts, and follow structured development workflows. Overall, version control significantly improved project maintainability and strengthened my practical knowledge of collaborative software engineering practices.

4.4.1 Git Basics and Local Version Control

The initial stage of learning Git involved understanding how version control helps manage changes in software development projects. During my internship, I learned the fundamentals of Git and how it enables developers to maintain different versions of project files systematically. Since the Automated Podcast Transcription and Topic Segmentation project involved continuous feature additions and modifications, version control became an important part of maintaining project stability and organization.

My learning journey began with installing Git and configuring the local development environment. I learned how to initialize a Git repository using the `git init` command, which allowed me to start tracking project files and maintain a version history. I also became familiar with configuring user information such as username and email, which helped in identifying project contributions through commit history.

I gained hands-on experience with essential Git commands that formed the foundation of local version control. The `git init` command was used to create a new Git repository for the project, enabling version tracking for all files and directories involved in development. The `git status` command helped me identify newly added, modified, or deleted files in the working directory, providing a clear understanding of which files were ready to be committed. Before saving changes permanently, I used `git add` to move updated files into the staging area, allowing selective inclusion of modifications in each commit.

The `git commit` command enabled me to save project changes with meaningful commit messages describing implemented updates. For example, commits were created after adding podcast upload functionality, improving transcript generation, or integrating topic segmentation features. Additionally, by using the `git log` command, I could track the sequence of commits and review previous versions of the project, which helped in monitoring progress and revisiting earlier implementations whenever required.

Working with Git locally improved my confidence in managing source code efficiently. I learned how to track changes systematically, restore previous versions when needed, and maintain organized development workflows. Local version control also reduced the risk of accidental data loss and ensured that experimentation with new features could be performed safely without affecting the overall project structure. Overall, understanding Git basics laid a strong foundation for professional software development practices and prepared me for collaborative version control using GitHub.

4.4.2 Collaborative Development Using GitHub

GitHub played an important role in managing and organizing the development process of

my internship project. After learning the basics of Git, I used GitHub as a cloud-based platform to store project repositories, maintain version history, and securely manage project files. Since the Automated Podcast Transcription and Topic Segmentation project involved multiple modules such as podcast upload, FasterWhisper integration, Natural Language Processing (NLP), Flask backend APIs, and dashboard development, GitHub helped in maintaining a structured and centralized workflow throughout development.

One of the primary benefits of GitHub during the internship was its ability to maintain remote backups of the project. After completing individual development tasks, I regularly pushed updates to the GitHub repository using commands such as `git push`. This ensured that all project changes were securely stored online and could be accessed from different systems whenever required. Similarly, using `git pull` allowed me to synchronize the latest updates and maintain consistency in the project codebase.

GitHub also improved code organization through repositories and branch management. Instead of directly modifying the main project files, I learned to create separate branches for experimenting with new functionalities or testing improvements. For example, individual updates such as transcript display enhancements, topic segmentation improvements, API optimization, and frontend dashboard modifications could be developed independently without affecting the stable version of the application. After successful testing, these changes were merged into the main branch to maintain project stability.

Overall, using GitHub during the internship significantly enhanced my software development workflow by improving project organization, version tracking, code security, and collaboration practices. It provided valuable exposure to industry-standard development methodologies and strengthened my confidence in handling real-world software projects effectively.

Furthermore, GitHub supported better collaboration and knowledge sharing by enabling easy access to project code and updates. Even while working individually, the platform simulated a real-world team environment where code can be reviewed, updated, and improved continuously. It allowed me to understand how developers contribute to shared projects, follow coding standards, and maintain consistency across different modules. This exposure helped me develop a more disciplined approach to coding, encouraged the use of best practices, and prepared me for working in collaborative software development environments in the future.

4.4.3 Real-World Development Experience

The practical implementation phase of the internship provided me with valuable real-world software development experience by allowing me to work on the Automated Podcast Transcription and Topic Segmentation project. This stage involved integrating Artificial Intelligence models, backend APIs, frontend interfaces, and database workflows into a single functional system. Through this experience, I gained a better understanding of how different technologies interact to solve practical problems and deliver meaningful outputs to users.

One of the most important aspects of the project was implementing a complete workflow for podcast analysis. The system was designed to process uploaded podcast files, convert audio into text using the FasterWhisper model, and perform topic segmentation using Natural Language Processing (NLP) techniques. I worked on integrating these modules through Flask APIs, ensuring smooth communication between the frontend and backend components. This helped me understand the importance of modular development, API handling, and efficient data flow in intelligent applications.

A major learning experience involved handling dynamic data processing and frontend-backend interaction. I implemented features that allowed users to upload podcast files, initiate processing, and retrieve generated transcripts, segmented topics, keywords, and summaries in real time. This improved user experience and taught me how modern web applications manage asynchronous operations and real-time interactions.

Overall, the real-world development experience significantly improved my technical skills, problem-solving ability, and confidence in building intelligent software systems. It strengthened my understanding of full-stack development, Artificial Intelligence integration, API development, and collaborative software engineering practices. The experience also prepared me to work on future real-world projects involving emerging technologies and practical problem-solving approaches.

Chapter 5

INTERNSHIP PROJECT CONTRIBUTION

After successfully completing the training phase, the next stage of my internship involved applying the acquired technical knowledge to the development of a real-world project titled “Automated Podcast Transcription and Topic Segmentation.” This phase marked a significant transition from theoretical learning to practical implementation, where I actively contributed to designing and building an Artificial Intelligence-based application capable of analyzing podcast content. My initial focus was on understanding the project objectives, application workflow, and the technologies involved in the development process. I thoroughly explored how audio data moved through different stages of the system—from podcast upload and preprocessing to transcription, Natural Language Processing (NLP), topic segmentation, and final dashboard presentation.

The project was developed using a combination of technologies including Python, Flask, FasterWhisper, Natural Language Processing libraries, audio processing tools, and frontend technologies for dashboard integration. Understanding how these components interacted was essential before beginning implementation. I carefully studied the architecture of the system, the role of backend APIs, the communication between frontend and Flask services, and how transcription and text analysis modules worked together to generate meaningful outputs. This understanding helped me develop a structured approach toward building and improving individual project modules.

One of the first practical tasks involved developing and integrating core functionalities required for podcast analysis. This included implementing podcast upload mechanisms, handling audio preprocessing, integrating the FasterWhisper model for speech-to-text conversion, and developing backend APIs to manage processing workflows. These tasks exposed me to real-world software development challenges, such as handling large audio files, processing delays, error handling, and maintaining application stability.

As development progressed, I contributed to enhancing the intelligence and usability of the system by implementing Natural Language Processing techniques for topic segmentation, keyword extraction, and summarization. These features transformed lengthy podcast audio into structured and organized textual information, making it easier for users to understand podcast content quickly. I also worked on integrating these processed outputs into a centralized dashboard where users could access transcripts, segmented topics,

summaries, and keywords in a clear and user-friendly format.

Beyond implementation, this phase also provided valuable exposure to debugging, testing, and iterative improvement. I continuously tested Flask APIs, validated transcription quality, resolved integration issues between frontend and backend components, and optimized workflows for smoother performance. Every task required a balance of technical understanding, logical thinking, and problem-solving ability. The following sections explain each major project contribution in detail, describing the tasks performed, implementation process, challenges encountered, and the practical knowledge gained throughout the internship experience.

5.1 Initial Product Understanding and System Analysis

The first step after completing the training phase was gaining a complete understanding of the Automated Podcast Analysis System and its workflow. Before beginning implementation, I focused on analyzing the project requirements, identifying user needs, understanding the data flow, and studying how Artificial Intelligence techniques could be integrated to solve real-world podcast consumption challenges. Since podcasts are generally long-form and unstructured audio content, users often face difficulties in revisiting important discussions, identifying major topics, and extracting meaningful insights quickly. Therefore, the main objective of the system was to automate podcast transcription, topic segmentation, and content summarization to improve accessibility and user understanding.

To better understand the project architecture, I analyzed how different modules of the system interacted with each other. The application followed a modular workflow where uploaded podcast audio first passed through preprocessing stages before being sent to the FasterWhisper model for speech-to-text conversion. After transcript generation, Natural Language Processing (NLP) techniques were applied to identify topic boundaries, extract meaningful keywords, and generate concise summaries. Finally, the processed results were displayed through an interactive dashboard integrated with the Flask backend APIs. Understanding this end-to-end workflow helped me visualize how multiple technologies worked together to transform raw podcast audio into structured and meaningful information.

From a technical perspective, the project architecture consisted of several interconnected components including Flask for backend API handling, FasterWhisper for transcription, NLP libraries for text processing, and frontend technologies for dashboard visualization. I studied how Flask routes handled requests for podcast uploads, transcript

generation, topic extraction, and result retrieval. The API-based architecture ensured smooth communication between the frontend and backend components, allowing users to upload files and receive processed insights dynamically. I also analyzed how audio files were managed and how processed data was stored and retrieved efficiently during the workflow.

The project involved several major modules, including podcast upload and validation, automated transcription, transcript segmentation, topic identification, summary generation, keyword extraction, and dashboard presentation. I carefully studied how each module contributed to the overall functionality of the application and how they depended on one another for smooth execution. Particular attention was given to understanding how FasterWhisper processed audio efficiently and how NLP techniques improved the readability and organization of transcript data.

In addition to technical understanding, I evaluated the user interaction flow within the application. The dashboard was designed to provide a seamless user experience where users could upload podcast files, monitor processing, read transcripts, access topic-wise segmented content, and review summarized insights without requiring technical expertise. This analysis helped me understand how usability and functionality needed to work together to make the application practical for real-world users.

Overall, this system analysis phase played an essential role in preparing me for practical development tasks during the internship. It gave me a clear understanding of the project architecture, data flow, backend logic, and AI processing pipeline. More importantly, it helped me understand how intelligent software systems are designed to solve real-world problems by integrating Artificial Intelligence, backend services, and user-friendly interfaces into a single scalable application.

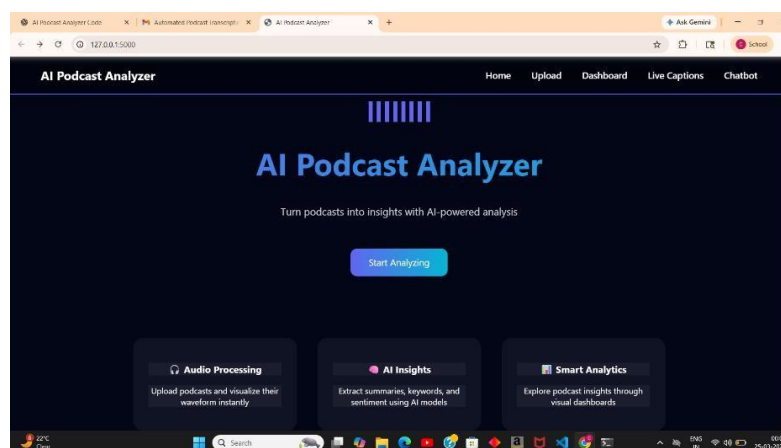


Fig 5.1: System Home of Automated Podcast Analysis Website

The interface provides navigation options such as Upload, Dashboard, Live Captions, and Chatbot. Users can initiate podcast analysis through a simple and user-friendly interface. The homepage highlights the system's core capabilities, including audio processing, AI insights, and smart analytics.

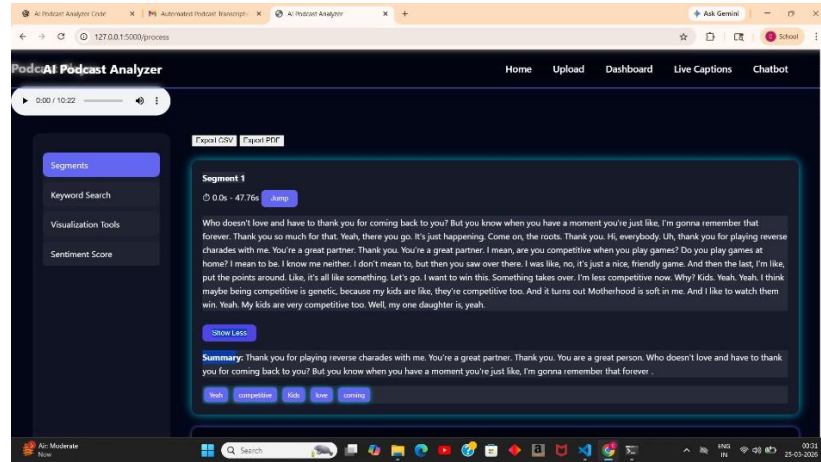


Fig 5.2: Workflow of Podcast Processing and Dashboard

Fig 5.2 shows the dashboard displaying processed podcast analysis results.

The system provides transcript segmentation, keyword extraction, summaries, and sentiment analysis. Users can navigate podcast segments and review important discussion points efficiently. The dashboard also supports export functionality and interactive visualization of podcast insights.

5.2 Frontend Console Cleanup and API Error Resolution

One of the initial technical tasks undertaken during the development of the Automated Podcast Transcription and Topic Segmentation system was resolving frontend issues and API-related errors to ensure smooth communication between the dashboard and the Flask backend. Since the project involved multiple integrated modules such as podcast uploading, transcription, summarization, topic segmentation, and chatbot interaction, maintaining a stable frontend environment became essential for seamless user experience. During testing, several issues such as API response mismatches, file upload failures, delayed rendering, and improper frontend validations were identified and systematically resolved.

A major focus area was debugging frontend console warnings and fixing API integration issues occurring during communication with Flask services. Several errors originated due to incorrect API endpoint handling, improper response parsing, and

asynchronous request failures while retrieving transcripts, summaries, and segmented podcast topics. To resolve these issues, I carefully reviewed API request structures, verified JSON response formats, and implemented proper error-handling techniques using try-catch mechanisms and conditional rendering. This helped prevent application crashes and ensured stable communication between the frontend interface and backend processing modules.

Additionally, file upload validation mechanisms were improved to handle unsupported formats, missing audio files, and interrupted uploads more effectively. The frontend was updated to display meaningful messages to users during upload failures or backend processing delays. Loading indicators and status messages were also introduced to improve transparency during longer transcription processes, particularly while handling lengthy podcast files through the FasterWhisper model.

Furthermore, debugging browser console logs helped identify performance bottlenecks and UI inconsistencies affecting dashboard responsiveness. By resolving these frontend and backend communication issues, the overall reliability, usability, and maintainability of the podcast analysis platform significantly improved. This task enhanced my understanding of frontend-backend debugging, REST API integration, asynchronous processing, and real-world application troubleshooting.

5.3 Backend Code Optimization Through Separation of Concerns

As part of improving the maintainability, scalability, and overall performance of the application, one of my major contributions focused on optimizing the backend code structure by enforcing the Separation of Concerns (SoC) principle. Initially, the backend logic was relatively monolithic, with route handlers tightly coupled with business logic and direct database operations. This not only made the code harder to debug and extend but also increased the risk of One of the major technical contributions during the internship involved optimizing the backend architecture of the Automated Podcast Analysis system to improve scalability, maintainability, and processing efficiency. Since the application handled computationally intensive tasks such as speech-to-text transcription, Natural Language Processing (NLP), summarization, and keyword extraction, maintaining an organized and modular backend structure became essential. To achieve this, I focused on improving the Flask backend by separating concerns and optimizing the overall processing workflow.

Initially, the backend logic involved tightly coupled processing functions, making it difficult to maintain or extend features efficiently. To improve this structure, I

modularized the Flask application into separate functional layers, where routes handled incoming requests, processing modules managed podcast transcription and segmentation logic, and helper utilities handled reusable operations such as text preprocessing, file management, and result formatting. This modular approach significantly improved code readability and simplified debugging.

The FasterWhisper transcription process was optimized to improve performance while handling long-duration podcast audio files. File processing logic was structured to efficiently manage uploaded podcast files, reducing redundant computations and ensuring proper handling of temporary resources. Similarly, Natural Language Processing components for topic segmentation, keyword extraction, and summarization were organized into independent modules, allowing each processing stage to operate systematically without affecting other functionalities.

To ensure backend reliability, extensive API testing was performed using Postman and browser-based testing environments. Various edge cases such as invalid audio files, unsupported formats, empty transcripts, and long processing times were tested to validate application stability. This optimization exercise deepened my understanding of backend architecture design, modular Flask development, API handling, and AI-powered processing pipelines used in intelligent applications.

5.4 Flask Middleware and Audio Processing Management

Managing audio uploads and processing requests efficiently was a critical requirement for the Automated Podcast Analysis system. To ensure smooth request handling and reliable execution of backend operations, I worked on implementing processing management mechanisms within the Flask application. Although Flask does not rely on middleware in the same manner as Express.js, several request validation and processing layers were incorporated to handle podcast uploads securely and efficiently.

The first step involved implementing file validation mechanisms during podcast upload. Before initiating transcription, uploaded audio files were checked for supported formats, file size limitations, and proper file integrity. This prevented invalid or corrupted files from entering the processing pipeline and reduced unnecessary computational overhead during AI-based transcription.

Request handling logic was also improved to manage processing delays more effectively. Since speech-to-text generation using FasterWhisper can be time-intensive for lengthy podcasts, processing status updates were introduced to inform users about current execution stages. This included validating requests before transcription, handling

temporary storage for uploaded audio files, and returning structured responses once processing was completed.

Error handling played a significant role in maintaining backend stability. Exception handling mechanisms were introduced to manage issues such as missing files, processing interruptions, model execution failures, or NLP processing errors. Appropriate HTTP response codes and meaningful messages were returned to ensure better frontend communication and improved user experience.

Overall, this implementation enhanced my understanding of request lifecycle management, backend validation, file handling, and reliable AI system execution using Flask APIs. It highlighted the importance of structured request processing for building scalable and production-ready intelligent applications.

5.5 Podcast Upload, AI Processing and Data Flow Implementation

One of the most significant contributions during the internship involved implementing the complete podcast processing workflow, which formed the backbone of the Automated Podcast Transcription and Topic Segmentation system. Since the primary objective of the application was to transform lengthy podcast audio into structured and searchable insights, a robust data flow mechanism was designed to ensure smooth interaction between frontend interfaces, Flask APIs, AI processing modules, and output visualization components.

The process began with the implementation of podcast upload functionality through the dashboard interface. Users were provided with an intuitive upload module where podcast audio files could be submitted directly into the system. Appropriate validations were added to verify supported file formats and prevent invalid uploads. Once the podcast was uploaded successfully, the frontend triggered Flask APIs to initiate automated processing. This upload-to-processing pipeline ensured seamless communication between the user interface and backend services.

After successful upload, the FasterWhisper speech recognition model was integrated into the processing pipeline to convert audio content into textual transcripts. The generated transcript formed the foundation for subsequent Natural Language Processing (NLP) operations. Since podcasts often contain long and unstructured discussions, the transcript data was further processed through topic segmentation algorithms to identify meaningful discussion boundaries and organize the content into understandable sections.

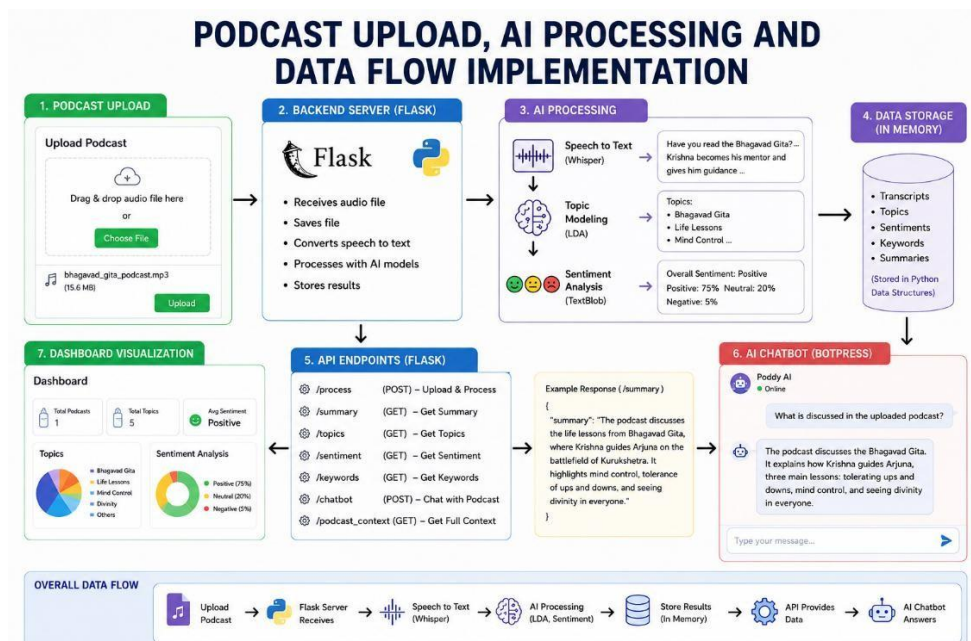


Fig 5.3: Podcast Upload, AI Processing and Data Flow Implementation

In addition to segmentation, keyword extraction and text summarization techniques were implemented to generate concise summaries for individual podcast segments. These outputs helped users quickly understand important discussion points without reading complete transcripts. Extracted keywords also improved content discoverability and enabled faster navigation through podcast topics. The processed results were dynamically sent back to the frontend dashboard through Flask APIs, where users could view transcripts, segmented discussions, and summarized insights in an organized format.

This implementation helped me understand how multiple technologies—including Flask, FasterWhisper, NLP models, and frontend APIs—work together to create an end-to-end intelligent software solution. It also strengthened my knowledge of data pipelines, asynchronous processing, AI model integration, and full-stack system coordination.

5.6 Real-Time Caption Generation and Chatbot Integration

One of the advanced functionalities implemented during the internship was the integration of real-time caption generation and chatbot interaction to enhance podcast accessibility and user engagement. Since podcasts are often lengthy and difficult to navigate manually, introducing dynamic captions and an intelligent chatbot significantly improved the usability of the application by enabling users to interact with podcast content more effectively.

The real-time caption generation module was developed to display transcript content dynamically as podcast audio played. Using FasterWhisper’s speech recognition

capabilities, audio segments were processed incrementally to generate timestamp-based text outputs. These captions were synchronized with podcast playback, enabling users to follow conversations in real time without relying solely on audio. This functionality improved accessibility, especially for users who preferred reading while listening or revisiting specific sections of discussions.

In parallel, a chatbot interface was integrated to allow users to interact intelligently with podcast content. The chatbot was designed to answer questions based on uploaded podcast transcripts, enabling users to quickly retrieve important information without manually scanning lengthy text.

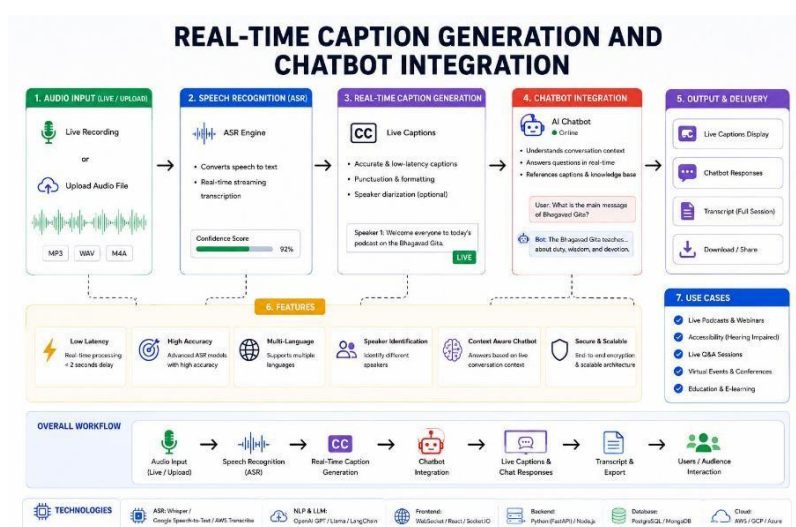


Fig 5.4: Real-Time Caption Generation and Chatbot Integration

Considerable effort was also invested in ensuring seamless frontend-backend communication for both caption updates and chatbot interactions. API endpoints were optimized to minimize response delays and maintain smooth data retrieval. Error-handling mechanisms were implemented to manage missing transcript data, interrupted requests, or incomplete podcast processing scenarios.

This feature provided valuable practical experience in integrating AI-driven conversational interfaces, managing real-time data updates, and improving user interaction within intelligent systems. It also deepened my understanding of accessibility-focused application design and real-world AI integration techniques.

5.7 Transcript, Summary and Analytics Report Functionality

Another important contribution during the internship was the implementation of transcript visualization, summary generation, and analytics reporting features within the podcast dashboard. Since podcasts can span several hours, manually reviewing lengthy discussions

is often time-consuming. To solve this challenge, structured reporting functionalities were incorporated into the application to transform raw podcast content into organized and meaningful insights.

The first major functionality involved displaying generated podcast transcripts in an organized and user-friendly format. After transcription using FasterWhisper, the complete textual output was presented through the dashboard, allowing users to review podcast discussions efficiently. Timestamp-based segmentation was implemented to improve navigation, enabling users to quickly access specific sections of the podcast without replaying the entire audio.

In addition to transcripts, automatic summarization functionality was introduced using Natural Language Processing techniques. Each identified topic segment was summarized into concise text blocks highlighting the most important discussion points. This significantly reduced the effort required to understand lengthy conversations and improved content accessibility for users seeking quick insights. Keyword extraction methods were also integrated to identify major concepts discussed throughout the podcast, helping users recognize important discussion themes instantly.

To improve usability further, analytics and export functionalities were incorporated into the dashboard. Processed podcast data—including transcripts, segmented topics, summaries, and extracted keywords—could be organized into downloadable formats for future reference and documentation. This feature increased the practical usability of the system for students, researchers, professionals, and podcast listeners interested in structured knowledge extraction.

Through this implementation, I gained hands-on experience in designing information-rich dashboards, managing structured AI outputs, and presenting complex data in an accessible manner. This task strengthened my skills in frontend integration, NLP visualization, and intelligent report generation systems.

5.8 Dashboard Styling and User Interface Enhancements

An intuitive and visually appealing user interface plays a vital role in improving user interaction and accessibility in intelligent applications. During the internship, I actively contributed to enhancing the dashboard interface of the Automated Podcast Transcription and Topic Segmentation system to ensure that podcast insights were presented in a clean, organized, and user-friendly manner. Since the application handled multiple outputs such as transcripts, segmented topics, summaries, keywords, captions, and chatbot interactions, designing a structured interface became essential for improving usability and navigation.

The dashboard interface was carefully improved to support smooth interaction between users and the backend AI system. Various UI components were optimized to create a better visual hierarchy and improve readability of processed podcast information. Dedicated sections were designed for podcast uploads, transcript viewing, topic segmentation, generated summaries, and keyword analytics. These improvements ensured that users could easily navigate between functionalities without confusion, thereby enhancing the overall experience of interacting with podcast content.

Special attention was given to responsive design and accessibility. The dashboard layout was adjusted to work efficiently across different screen sizes and devices, ensuring consistency in user experience. Dynamic rendering techniques were implemented to display newly processed podcast outputs without requiring page refreshes. Loading indicators, progress messages, and status updates were added to improve transparency during long-running AI processes such as transcription and segmentation.

Furthermore, several styling improvements were introduced to improve visual consistency and reduce interface clutter. Typography, spacing, alignment, and content grouping were refined to make transcripts and summaries easier to read. Interactive components such as buttons, navigation sections, upload fields, and chatbot interfaces were designed to improve engagement and usability. These enhancements significantly contributed to making the application more practical and user-friendly.

Through this experience, I gained a deeper understanding of frontend design principles, responsive dashboard development, UI optimization, and user-centered software design. It also highlighted the importance of balancing functionality with visual simplicity while building intelligent applications.

5.9 Version Control and Code Management Using GitHub

Version control played an important role throughout the development of the Automated Podcast Transcription and Topic Segmentation system. Since the project involved continuous improvements in podcast upload functionality, FasterWhisper transcription, NLP processing, dashboard development, chatbot integration, and API optimization, maintaining organized and secure project management became essential. During the internship, Git and GitHub were extensively used to track development progress, manage code versions, and maintain stability during feature additions .

Git branching and version tracking also helped manage experimentation and feature implementation more efficiently. Instead of directly modifying the stable project structure, separate branches were used to test improvements related to podcast upload

handling, transcript generation, topic segmentation logic, chatbot integration, and frontend dashboard modifications. Once features were successfully tested and validated, they were merged into the primary branch to maintain application stability.

Additionally, maintaining meaningful commit messages allowed efficient tracking of development progress.

Overall, working with Git and GitHub strengthened my understanding of professional software development workflows, collaborative coding standards, branch management, and secure version tracking. It provided practical exposure to industry-standard development practices and improved my confidence in managing real-world software projects systematically.

5.10 Real-Time Bug Fixes and Feature Enhancements

During the later stages of the internship, a significant portion of my responsibilities involved identifying, analyzing, and resolving real-time bugs while continuously enhancing the overall functionality of the Automated Podcast Analysis System. As the project evolved into a fully integrated AI-powered platform, multiple modules needed to work together seamlessly. These modules included podcast upload and management, audio preprocessing, speech-to-text transcription, topic segmentation, summarization, keyword extraction, sentiment analysis, AI chatbot interaction, report generation, and database management. Since each component depended on the successful execution of previous processes, even minor issues could affect the entire workflow. Therefore, extensive testing, debugging, and optimization became critical to ensure system reliability, performance, and user satisfaction.

One of the primary challenges encountered during development was ensuring the smooth integration of different technologies and APIs. The system relied on several machine learning models, natural language processing libraries, and backend services that interacted with one another in real time. During testing, issues such as delayed API responses, incorrect transcript generation, incomplete data processing, and synchronization errors between modules were identified. These problems required careful analysis to determine their root causes. Debugging involved reviewing application logs, monitoring system behavior, validating API responses, and tracing data flow across different modules. By systematically examining each component, I was able to identify inconsistencies and implement appropriate solutions.

A major focus area was the transcription module, which converts podcast audio into text. Since podcasts vary significantly in audio quality, speaking speed, accents, and background

noise levels, the transcription process occasionally produced inaccuracies or incomplete results. To address these issues, multiple test cases were executed using podcasts of different durations and recording qualities. Error handling mechanisms were improved to detect transcription failures and automatically notify users when processing issues occurred. Additional validation checks were introduced to ensure that transcripts were generated correctly before proceeding to subsequent analysis stages. These improvements increased the reliability and consistency of the speech-to-text pipeline.

Another critical area involved debugging the topic segmentation module. Topic segmentation plays an important role in dividing long podcast transcripts into meaningful sections based on discussion themes. During testing, certain transcripts produced inaccurate segment boundaries, causing unrelated topics to be grouped together. This affected downstream tasks such as summarization and chatbot responses. To improve segmentation accuracy, multiple experiments were conducted using different transcript samples and segmentation parameters. Algorithm adjustments and preprocessing enhancements were implemented to improve the detection of topic transitions. As a result, the generated topic sections became more coherent and meaningful, enhancing the overall quality of analysis.

Overall, this phase of the internship significantly strengthened my technical and analytical skills. It improved my ability to diagnose complex software issues, implement effective solutions, optimize system performance, and enhance user experience. The practical exposure to production-level debugging, quality assurance, feature refinement, and system optimization provided a deeper understanding of modern software engineering practices. These experiences contributed substantially to my professional growth and reinforced the importance of continuous improvement in building robust, scalable, and efficient AI-driven software solutions.

CONCLUSION

This internship project has been a highly valuable and practical learning experience, allowing me to develop and apply full-stack development skills in a real-world AI-based system. The primary focus of my work was the design and implementation of an AI-powered podcast processing platform, which includes podcast upload, transcription, content extraction, and backend workflow integration.

Throughout the project, I worked on multiple core modules such as podcast file upload handling, backend API development using Flask, frontend integration for user interaction, and structured data flow between services. I also contributed to debugging frontend issues, improving API communication, and ensuring smooth data transfer between the chatbot and backend system.

A significant part of my work involved integrating AI-based processing pipelines, where uploaded podcast content is extracted, processed, and converted into structured formats for further use. Additionally, I handled workflow improvements such as router-agent table handling, execution flow corrections, and system stability enhancements.

This project also helped me gain practical exposure to debugging API requests, managing system architecture, and improving integration between different services like frontend UI, backend APIs, and chatbot systems. I learned the importance of clean architecture, modular design, and efficient data flow in building scalable applications.

Overall, this internship project strengthened my technical knowledge in full-stack development and AI system integration. It provided hands-on experience in building a production-oriented application and significantly improved my problem-solving, debugging, and system design skills. This experience has prepared me with a strong foundation for future work in advanced software and AI-driven applications.

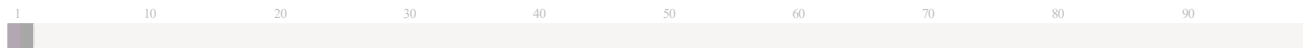
REFERENCES

- [1] Alec Radford et al., “Robust Speech Recognition via Large-Scale Weak Supervision,” in Proceedings of ICML, 2023..
- [2] Python Software Foundation, “Python Language Reference.” [Online]. Available: <https://docs.python.org/3/>
- [3] OpenAI, “Whisper Speech-to-Text Model Documentation.” [Online]. Available: <https://github.com/openai/whisper>
- [4] OpenAI, “OpenAI API Documentation.” [Online]. Available: <https://platform.openai.com/docs>
- [5] Ashish Vaswani et al., “Attention Is All You Need,” in Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [6] Hugging Face, “Transformers Library Documentation.” [Online]. Available: <https://huggingface.co/docs/transformers>
- [7] SentenceTransformers, “Sentence-BERT and Embedding Models Documentation.” [Online]. Available: <https://www.sbert.net>
- [8] TextBlob, “TextBlob Documentation.” [Online]. Available: <https://textblob.readthedocs.io>
- [9] YAKE, “Yet Another Keyword Extractor (YAKE) Documentation.” [Online]. Available: <https://github.com/LIAAD/yake>
- [10] Hugging Face, “BART Model for Text Summarization (DistilBART / BART variants).” [Online]. Available: <https://huggingface.co/facebook/bart-large-cnn>
- [11] NumPy Developers, “NumPy Documentation.” [Online]. Available: <https://numpy.org/doc>
- [12] Maarten Grootendorst, “BERTopic: Neural Topic Modeling with Transformer Embeddings,” 2022
- [13] WordCloud Library, “WordCloud for Python Documentation.” [Online]. Available: https://amueller.github.io/word_cloud

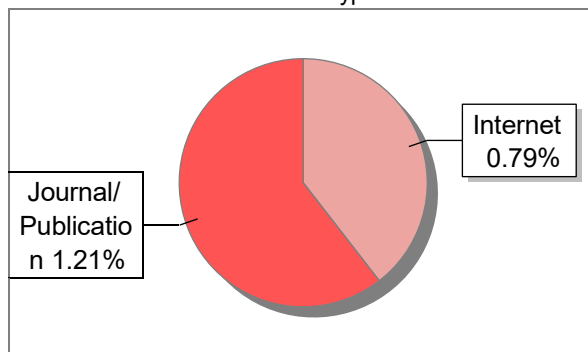
Submission Information

Author Name	Chintawar Aditi Arvind
Title	Updated internship report sample
Paper/Submission ID	5761784
Submitted by	rajurkar_am@mgmcen.ac.in
Submission Date	2026-05-30
Total Pages, Total Words	76, 20686
Document type	Project Report

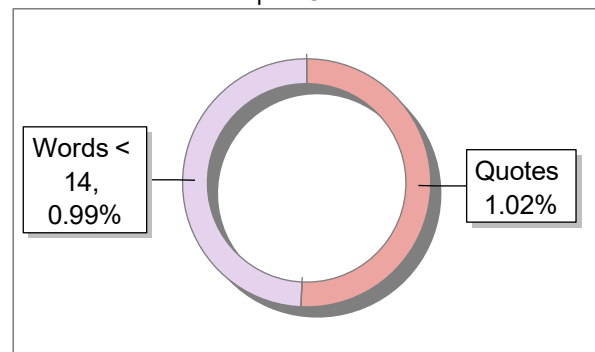
Result Information

Similarity **2 %**

Sources Type



Report Content



Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

